UNIVERSITI TEKNOLOGI MALAYSIA

Semester 2, Session 2025/2026

**SCSE2243 APPLICATION DEVELOPMENT I**

SECTION 01

## SOFTWARE DATA DESIGN

# UTM BORROW

*A Campus-Wide Circular Resource Sharing Platform*

| | |
|---|---|
| **Prepared For** | DR. MOHD YAZID BIN IDRIS |
| **Submission Date** | 26 - MAY 2026 |
| **Group Name** | Alpha |



| NAME | MATRIC NUMBER |
|---|---|
| MOHAMMED MOHSEN MOHAMMED ALSAKKAF | A23CS4026 |
| MUAZ IBNE AHMED | A23CS4062 |
| AHMAT MAHAMAT MOURDJI MOUSTAPHA | A24CS4053 |

*Faculty of Computing, Universiti Teknologi Malaysia*

# Table of Content


1. INTRODUCTION TO APPLICATION....4
1.1. Introduction....4
1.2. Problem Statement....4
1.3. The Proposed System....4
1.4. Technology and Tool Used....5
1.5. Functional Requirements....6
2. APPLICATION DIAGRAM....6
2.1. System Architecture Design....6
3. IDENTITY & USER MANAGEMENT SUBSYSTEM....7
3.1. AUTHENTICATION MODULE....7
3.1.1. Data Design....7
3.1.2. Data Dictionary & Description....8
• Entity: Users....8
• Entity: Password Recovery Tokens....9
3.1.3. User Interface Design....9
3.2. PROFILE & TRUST SCORE MODULE....13
3.2.1. Data Design....13
3.2.2. Data Dictionary & Description....13
• Entity: Users....14
• Entity: User Ratings....14
• Entity: Reports....14
3.2.3. User Interface Design....15
3.3. USER ACTIVITY DASHBOARD MODULE....16
3.3.1. Data Design....16
3.3.2. Data Dictionary & Description....17
• Entity: Users....17
• Entity: Transactions....18
• Entity: Items....18
3.3.3. User Interface Design....18
4. RESOURCE CATALOG & DISCOVERY SUBSYSTEM....20
4.1. ITEM LISTING & MANAGEMENT MODULE....20
4.1.1. Data Design....20
4.1.2. Data Dictionary & Description....20
• Entity: <Name>....20
4.1.3. User Interface Design....20
4.2. CATEGORY & CONDITION METADATA MODULE....21
4.2.1. Data Design....21
4.2.2. Data Dictionary & Description....21
• Entity: <Name>....21
4.2.3. User Interface Design....21
4.3. LOCATION-BASED FILTER MODULE....22
4.3.1. Data Design....22
4.3.2. Data Dictionary & Description....22
• Entity: <Name>....22
4.3.3. User Interface Design....22
5. TRANSACTION & HANDOVER SUBSYSTEM- Alsakkaf....23
5.1. REQUEST & APPROVAL WORKFLOW MODULE- Alsakkaf....23

5.1.1. Data Design ..... 23
5.1.2. Data Dictionary & Description ..... 24
- Entity: Users (Referenced — owned by Identity & User Management Subsystem) ..... 24
- Entity: Items (Referenced — owned by Identity & User Management Subsystem) ..... 24
- Entity: Transactions (Referenced — owned by Identity & User Management Subsystem) ..... 25
- Entity: Transaction_State_Logs ..... 26
- Entity: Notifications ..... 26

5.1.3. User Interface Design ..... 27
**5.2. QR VERIFICATION MODULE- Alsakkaf ..... 29**
5.2.1. Data Design ..... 29
5.2.2. Data Dictionary & Description ..... 30
- Entity: Users (Referenced — owned by Identity & User Management Subsystem) ..... 31
- Entity: Items (Referenced — owned by Identity & User Management Subsystem) ..... 31
- Entity: Transactions (Referenced — owned by Identity & User Management Subsystem) ..... 31
- Entity: QR_Tokens ..... 32
- Entity: Scan_Events ..... 33
- Entity: Lease_Cycles ..... 34

5.2.3. User Interface Design ..... 35
**5.3. COMMUNITY MODERATION & REPORTING MODULE - Alsakkaf ..... 38**
5.3.1. Data Design ..... 38
5.3.2. Data Dictionary & Description ..... 39
- Entity: Users (Referenced — owned by Identity & User Management Subsystem) ..... 39
- Entity: Items (Referenced — owned by Identity & User Management Subsystem) ..... 40
- Entity: Admins ..... 40
- Entity: Reports ..... 41
- Entity: Moderation_Actions ..... 42
- Entity: User_Suspensions ..... 43
- Entity: Notifications (Extended from Module 1) ..... 43

5.3.3. User Interface Design ..... 44

# 1. INTRODUCTION TO APPLICATION

## 1.1. Introduction

UTM Borrow is a campus-wide circular resource-sharing web application built for the Universiti Teknologi Malaysia (UTM) community. It provides a secure, centralized platform where verified students and staff can lend and borrow academic resources — laboratory tools, engineering equipment,
textbooks, electronics, and event clothing — instead of buying expensive, single-use items. Unlike unverified WhatsApp or Telegram groups, UTM Borrow enforces strict trust mechanics: access is restricted to official UTM email accounts, every loan is tracked through a formal state machine, and each physical handover and return is verified with a cryptographically signed QR code. The platform promotes campus sustainability by reducing post-semester waste and eases the financial burden on students who would otherwise purchase items they need only once.

## 1.2. Problem Statement

UTM students regularly face financial and logistical difficulties acquiring specialized, short-term academic resources. Many courses require high-cost, single-use items — scientific calculators, lab coats, drawing sets, formal attire, and specialized electronics — that are used for only one semester or project, fuelling a costly "buy-and-discard" culture and generating significant waste across residential colleges. Students currently rely on informal, unverified chat groups to share items, but these channels lack identity verification and any formal trust infrastructure, exposing users to scams, theft, and disputes over damaged or unreturned items. With no handover protocol, condition logging, or return tracking, lending is risky and owners are discouraged from sharing, preventing a sustainable campus circular economy from forming.

## 1.3. The Proposed System

UTM Borrow solves these problems with three integrated subsystems. The Identity & User Management
Subsystem** restricts access to verified UTM emails and maintains a peer-driven Trust Score and a personal activity dashboard. **The Resource Catalog & Discovery Subsystem** lets owners publish and
manage listings and lets borrowers find items through structured category, condition, and location-based filtering.
The **Transaction & Handover Subsystem** manages the full borrowing
lifecycle (Pending → Approved → Borrowed → Completed) with an atomic request lock, HMAC-SHA256
signed QR codes for tamper-proof handover and return, and a community moderation pipeline for reporting and enforcement. Real-time updates are pushed to every client over Server-Sent Events, so notifications, catalog changes, and transaction status updates appear instantly without refreshing.

## 1.4. Technology and Tool Used

UTM Borrow is a responsive web application built on a three-tier client–server architecture. The technologies below reflect the system as actually built and deployed.

- **Frontend:** React (Create React App) single-page application, styled with Tailwind CSS and animated with Framer Motion; communicates with the backend through the Axios HTTP client.
- **Backend:** FastAPI (Python) serving an asynchronous REST API (all routes under /api), which enforces validation, authentication, and the transaction state-machine logic.
- **Database:** MongoDB (accessed via the Motor async driver), a document-oriented store where every record is keyed by a unique string identifier.
- **Authentication & Security:** JSON Web Tokens (JWT, HS256, 7-day validity) for sessions, bcrypt password hashing, a regular-expression gate limiting registration to @utm.my / @graduate.utm.my, and TOTP multi-factor authentication for the admin moderation portal.
- **QR Verification:** HMAC-SHA256 signed, time-limited (24-hour) tokens in the form UTMB.<payload>.<signature>, verified server-side on every scan.
- **Real-time:** Server-Sent Events (SSE) pushing live updates from server to client over plain HTTP.
- **Deployment:** FastAPI backend on Render (Docker, python:3.12-slim, uvicorn); React build on Vercel; the two are wired through environment variables and a CORS policy.
- **Modeling & Project Management:** Visual Paradigm (UML), Figma (UI/UX prototypes), Jira and GitHub (task tracking and version control), and Generative AI (ChatGPT / Claude) as a development aid.

## 1.5. Functional Requirements

The platform spans three subsystems and nine modules. The system shall:

- **Identity & User Management**
  - Register and authenticate users only with valid UTM email addresses; hash all passwords.
  - Provide secure, token-based password recovery.
  - Maintain editable profiles and compute a dynamic 5-star Trust Score from completed transactions.
  - Let users rate counterparties and report users/items for moderation.
  - Present a real-time activity dashboard of items the user is lending and borrowing, with overdue alerts.
- **Resource Catalog & Discovery**
  - Let owners publish, edit, set visibility for, and remove their item listings.
  - Enforce controlled category and condition metadata on every listing.
  - Let borrowers search and filter the catalog by category, condition, college, and faculty.
- **Transaction & Handover**
  - Submit borrow requests with date validation and an atomic Available→Pending item lock.
  - Allow lender approval/rejection and request cancellation before handover.
  - Generate and verify HMAC-SHA256 signed QR codes for handover and return.
  - Log every transaction state change immutably; notify both parties in real time.
  - Provide an MFA-gated admin moderation portal to review reports and remove items or suspend users.

# 2. APPLICATION DIAGRAM

## 2.1. System Architecture Design

UTM Borrow follows a classic three-tier client–server architecture. The presentation tier is a React single-page application running in the browser, which renders all screens and talks to the server exclusively over HTTPS using the Axios client. The application tier is a FastAPI (Python) server exposing a REST API under /api; it handles authentication (JWT + bcrypt), input validation, the transaction state machine, QR signing and verification (HMAC-SHA256), and moderation logic. The data tier is a MongoDB database accessed through the asynchronous Motor driver, storing all users, items, transactions, and supporting collections as documents keyed by unique string identifiers.

Running alongside the request/response REST flow is a one-way Server-Sent Events (SSE) channel: when a server-side event occurs (a new notification, a catalog change, a transaction status update, a new chat message, or a moderation action), the server pushes it to the relevant connected clients so the interface updates live without polling. All state-changing operations still go through REST; SSE is push-only. In deployment, the React build is hosted on Vercel and the FastAPI server on Render, connected through environment variables and a CORS allowlist.

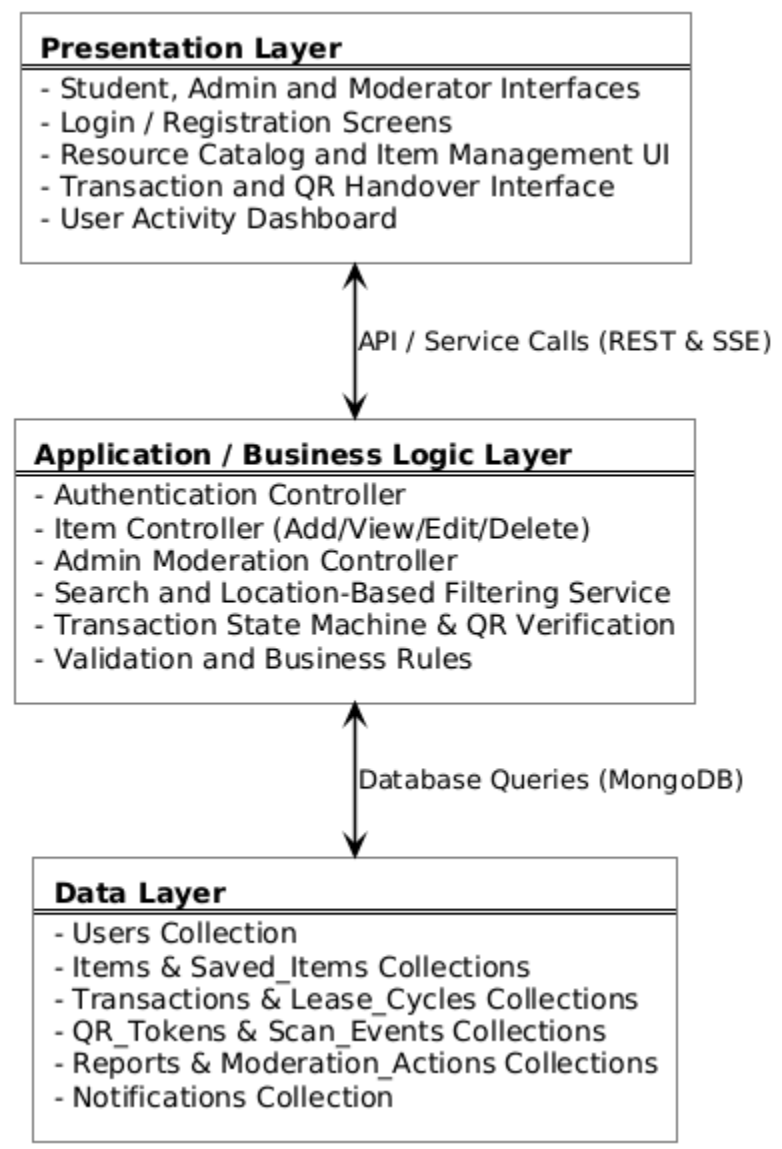



*Figure 2.1: Architecture Design of UTM Borrow*

# 3. IDENTITY & USER MANAGEMENT SUBSYSTEM

## 3.1. AUTHENTICATION MODULE

The Authentication Module serves as the primary security gateway for the UTM Borrow platform, enforcing identity verification by restricting access strictly to authorized UTM domains (@utm.my or @graduate.utm.my). This section outlines the data persistence strategies and interface states required to manage user registration, secure session initialization, and cryptographic password recovery.

### 3.1.1. Data Design

Figure 3.1.1 illustrates the entity-relationship mapping for the Authentication Module. The schema enforces a strict one-to-many dependency between the core `Users` entity and the volatile `Password_Recovery_Tokens` table, ensuring temporal constraints and referential integrity for account recovery actions.

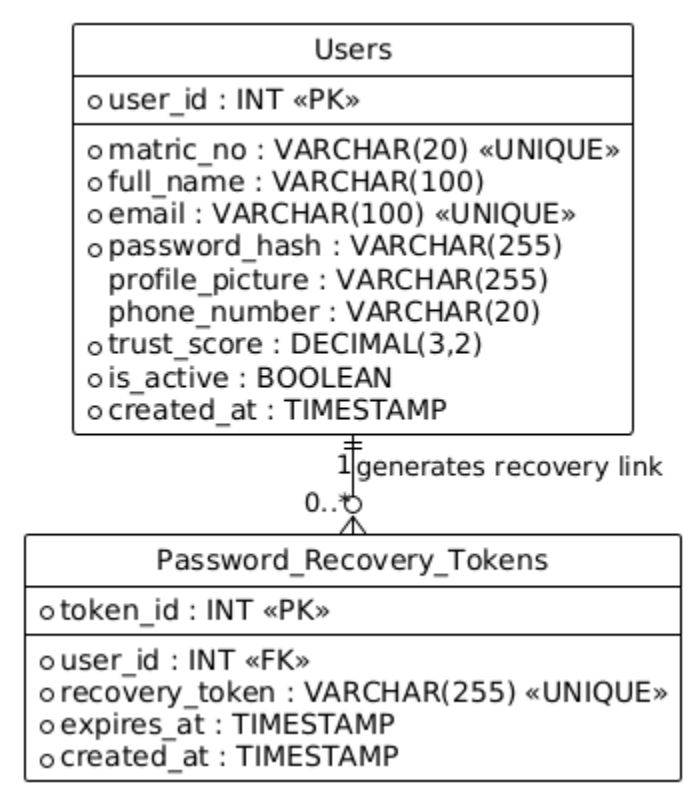



*Figure 3.1.1: Entity Relationship Design (ERD) for AUTHENTICATION MODULE*

## 3.1.2. Data Dictionary & Description

The following tables define the schema properties, data types, and constraint logic for the physical database tables governing user identity and authentication credentials.

- **Entity: Users**

| Attribute name | Data Type | Description |
|---|---|---|
| user_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| matric_no | VARCHAR(20) | NOT NULL, UNIQUE, UTM student matriculation number |
| full_name | VARCHAR(100) | NOT NULL, Display name shown to counterparties |
| email | VARCHAR(100) | NOT NULL, UNIQUE, UTM institutional email (must end in @utm.my or @graduate.utm.my) |
| password_hash | VARCHAR(255) | NOT NULL, Cryptographically hashed string for secure login |
| profile_picture | VARCHAR(255) | NULLABLE, URL or path to the stored image file |
| phone_number | VARCHAR(20) | NULLABLE, Contact number updated via profile management |
| trust_score | DECIMAL(3,2) | DEFAULT 5.00, Range 0.00-5.00, Updated after peer reviews |
| is_active | BOOLEAN | DEFAULT TRUE, FALSE indicates a suspended or deactivated account |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

*Table 3.1.2.1: Data Dictionary and Description of Users Table*

- **Entity: Password Recovery Tokens**

| Attribute name | Data Type | Description |
|---|---|---|
| token_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE CASCADE, Links token to the specific user requesting reset |
| recovery_token | VARCHAR(255) | NOT NULL, UNIQUE, Cryptographic token sent to user's inbox |
| expires_at | TIMESTAMP | NOT NULL, Exact time the token becomes invalid |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

*Table 3.1.2.2: Data Dictionary and Description of Password Recovery Tokens Table*

## 3.1.3. User Interface Design

The interface design for the Authentication Module prioritizes frictionless onboarding while maintaining strict input validation feedback. The following figures detail the layout constraints and error-state rendering for unauthorized domain attempts and credential mismatches.

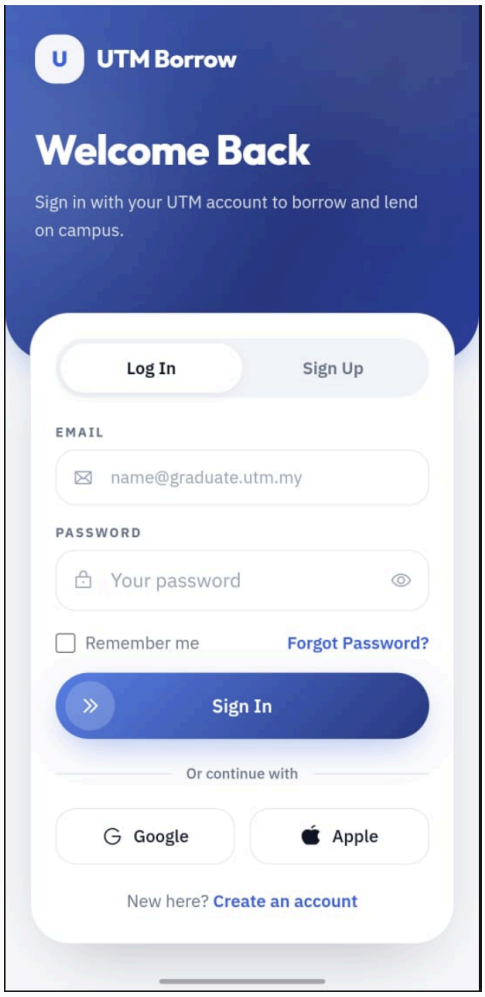



*Figure 3.1.3.1: User Interface for Login page for AUTHENTICATION MODULE*

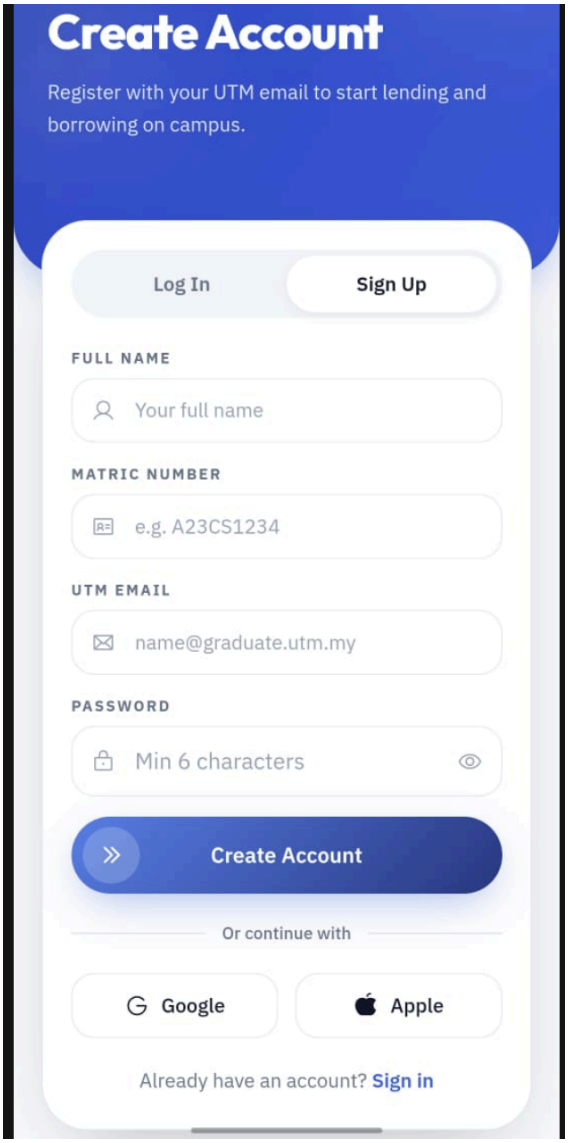



*Figure 3.1.3.2: User Interface for Register page for AUTHENTICATION MODULE*

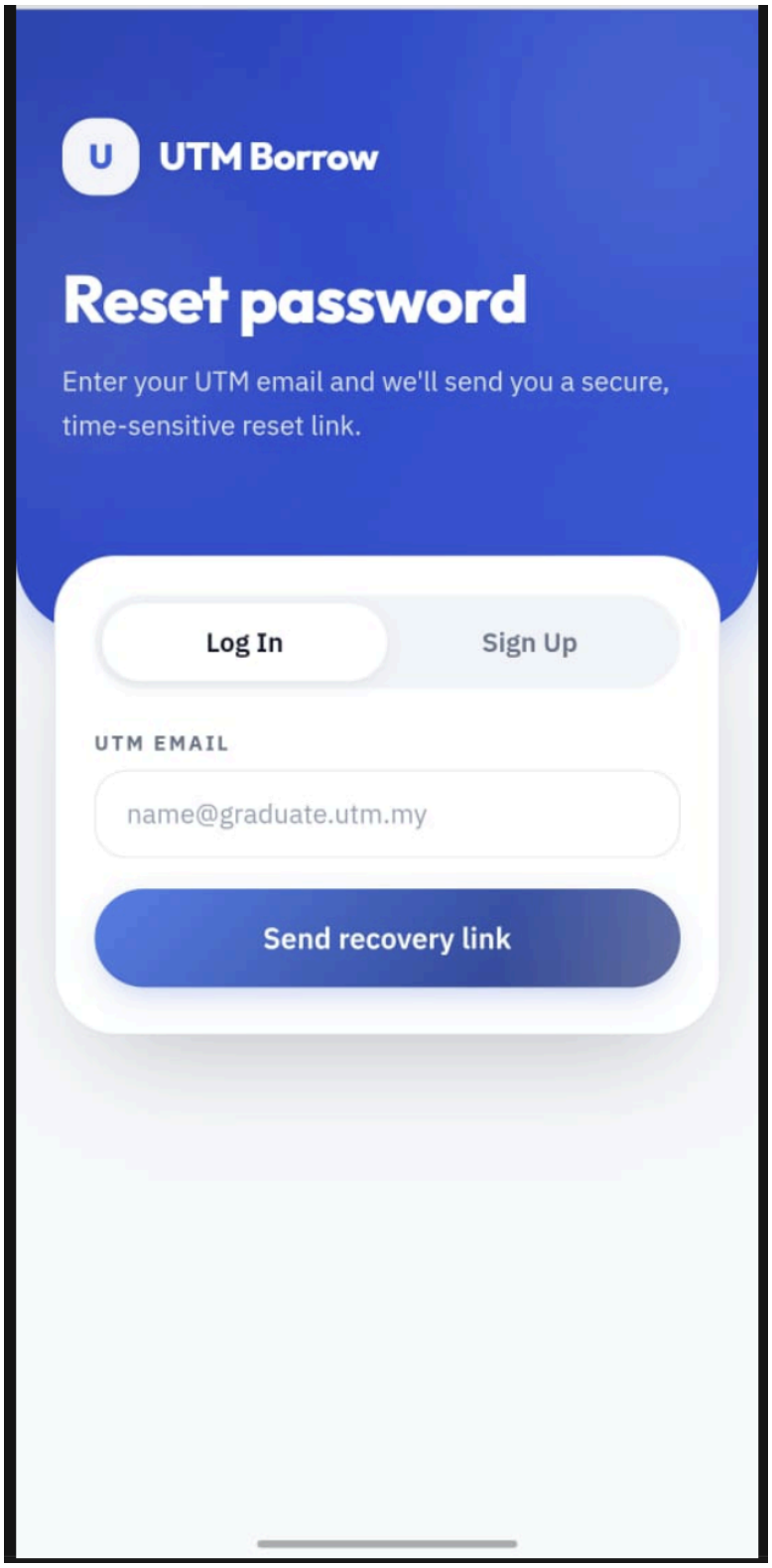



*Figure 3.1.3.3: User Interface for Password Resetting page for AUTHENTICATION MODULE*

# PROFILE & TRUST SCORE MODULE

This module governs the digital identity and reputation mechanics of the platform. It handles the persistence of user-modifiable profile metadata and the aggregation of peer-to-peer feedback to calculate a dynamic, immutable Trust Score, which is central to the platform's risk mitigation strategy.

## 3.1.4. Data Design

Figure 3.2.1 defines the structural relationships mapping the `Users` entity to transactional feedback records in the `User_Ratings` table, alongside dependencies on the Moderation subsystem's `Reports` entity to flag severe non-compliance.

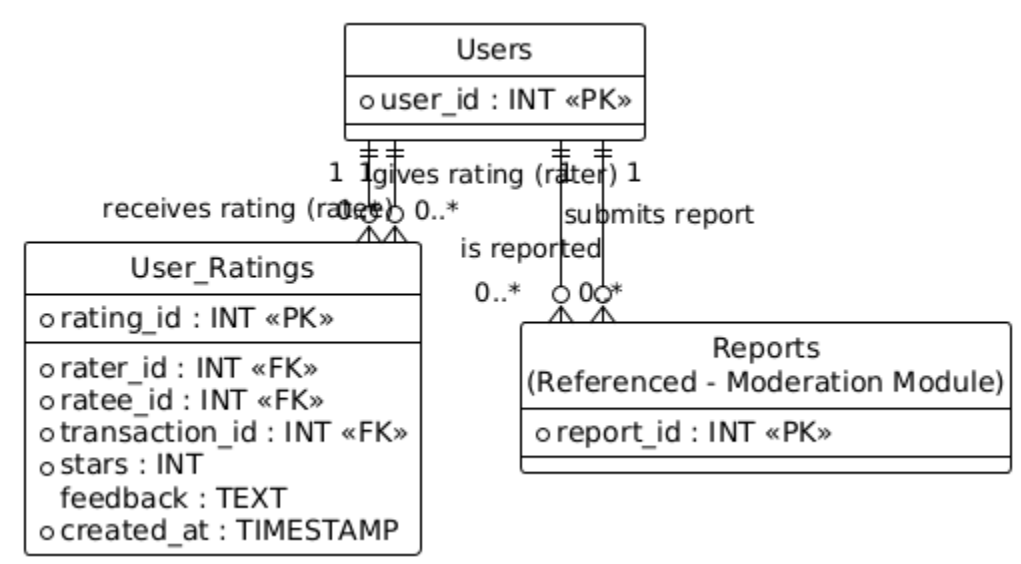



*Figure 3.2.1: Entity Relationship Design (ERD) for PROFILE & TRUST SCORE MODULE*

## 3.1.5. Data Dictionary & Description

The data dictionary below specifies the constraints for user-generated reputation data. The `User_Ratings` table establishes foreign key references to both the rater and ratee, guaranteeing that all Trust Score adjustments are strictly bound to verified, completed transactions.

- **Entity: Users**

| Attribute name | Data Type | Description |
|---|---|---|
| user_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| trust_score | DECIMAL(3,2) | DEFAULT 5.00, Dynamically recalculated average based on peer ratings |

*Table 3.2.2.1: Data Dictionary and Description of Users Table*

- **Entity: User Ratings**

| Attribute name | Data Type | Description |
|---|---|---|
| rating_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| rater_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, The student giving the rating |
| ratee_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, The student receiving the rating |
| transaction_id | INT | NOT NULL, Contextual link to the completed lease triggering this rating |
| stars | INT | NOT NULL, Integer constraint between 1 and 5 |
| feedback | TEXT | NULLABLE, Optional text review left by the rater |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

*Table 3.2.2.2: Data Dictionary and Description of User Ratings Table*

- **Entity: Reports**

| Attribute name | Data Type | Description |
|---|---|---|
| report_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| reporter_id | INT | NOT NULL, FOREIGN KEY → users.user_id, The student submitting the report |
| reported_user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, The offending user being flagged |
| report_category | ENUM | NOT NULL, Category of violation |

| | | |
|---|---|---|
| description | TEXT | NOT NULL, Detailed information regarding the violation |

*Table 3.2.2.3: Data Dictionary and Description of Reports Table*

## 3.1.6. User Interface Design

The following user interface design maps the interaction states for profile management and post-transaction feedback submission, including the conditional logic for rendering the 5-star rating dialog and the non-compliance reporting form.

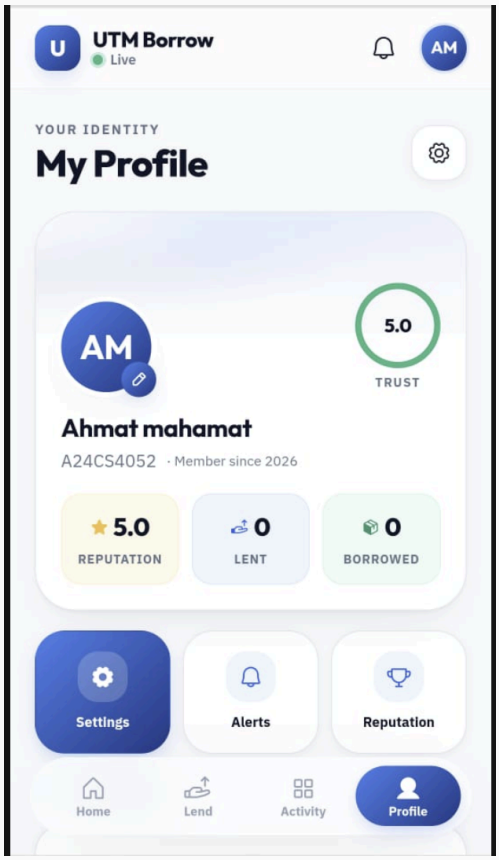



*Figure 3.2.3.1: User Interface for Profile page for PROFILE & TRUST SCORE MODULE*

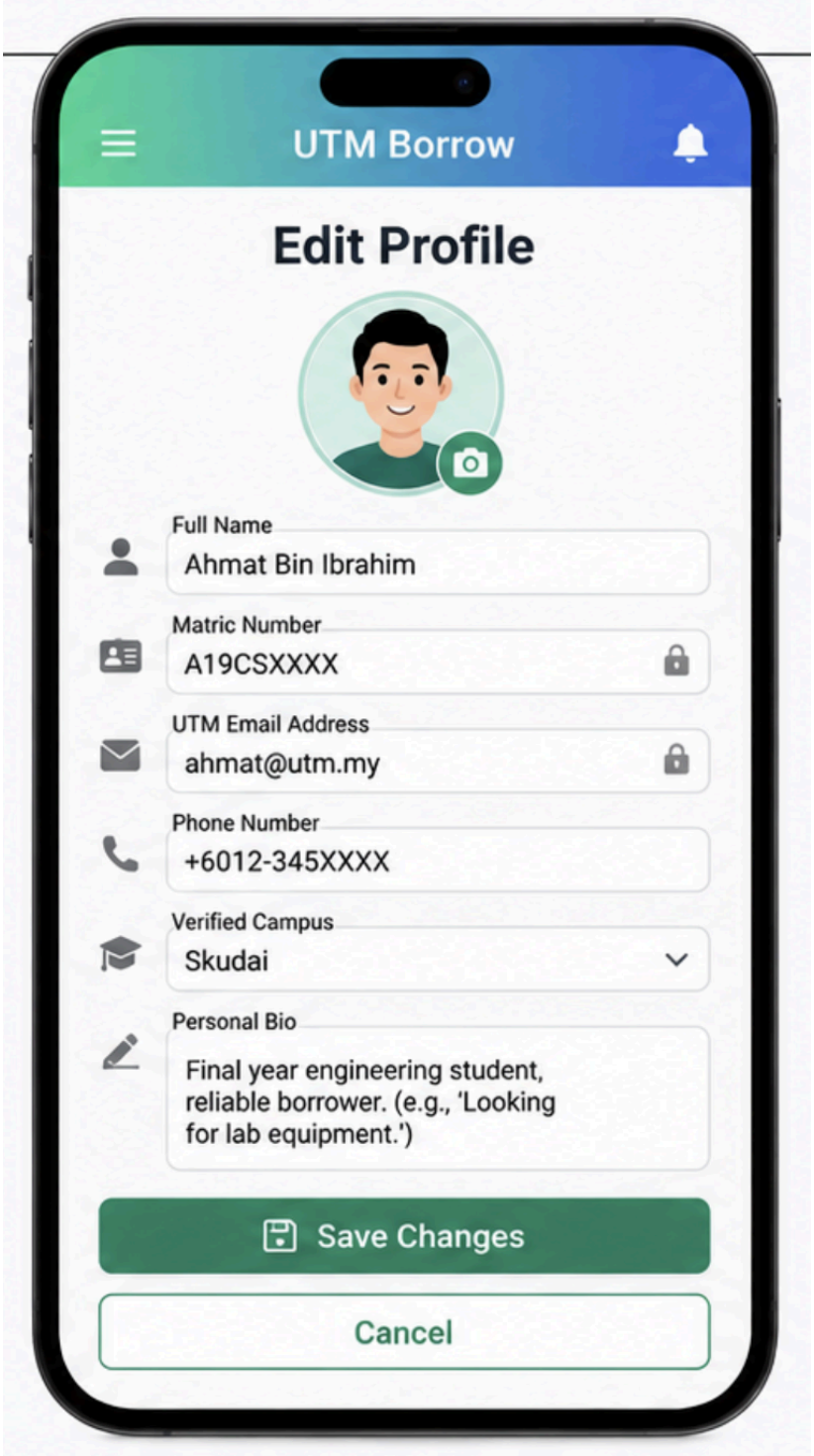



*Figure 3.2.3.2: User Interface for Profile Editing page for PROFILE & TRUST SCORE MODULE*

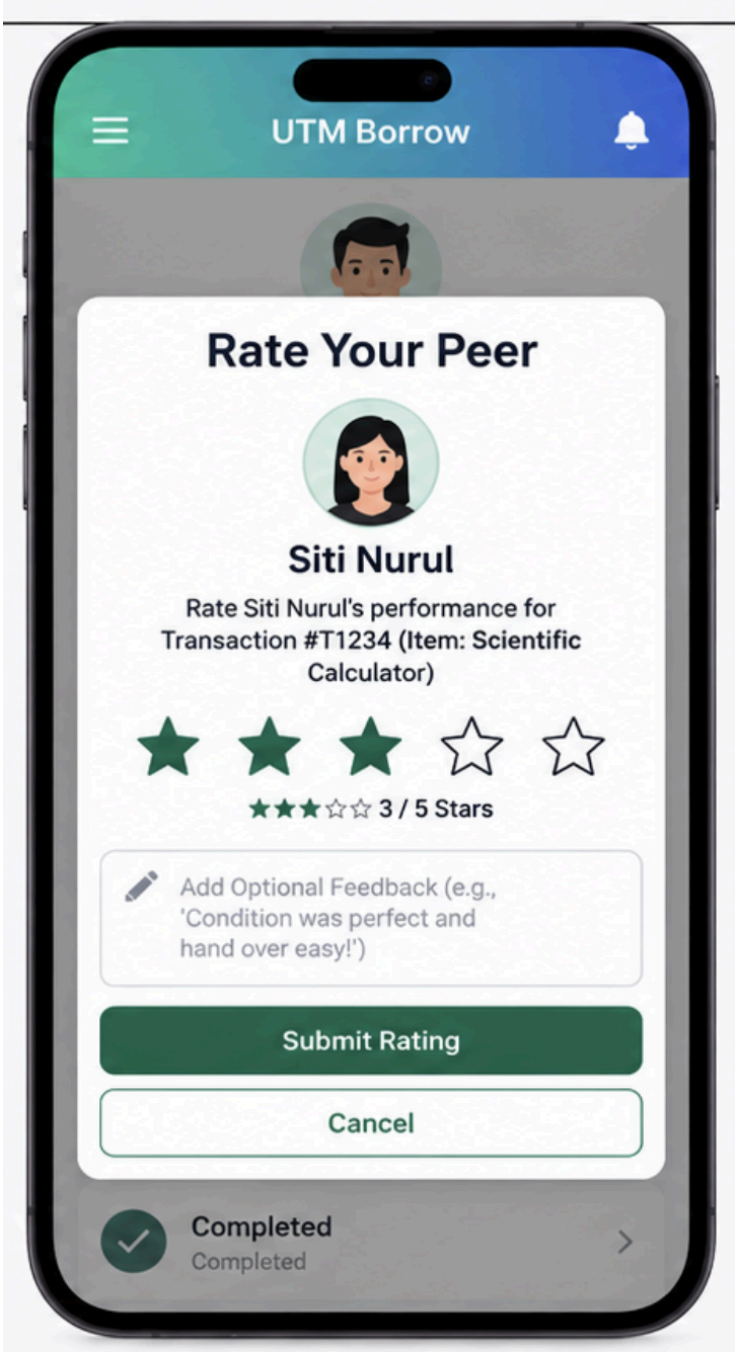



*Figure 3.2.3.3: User Interface for User Rating page for PROFILE & TRUST SCORE MODULE*

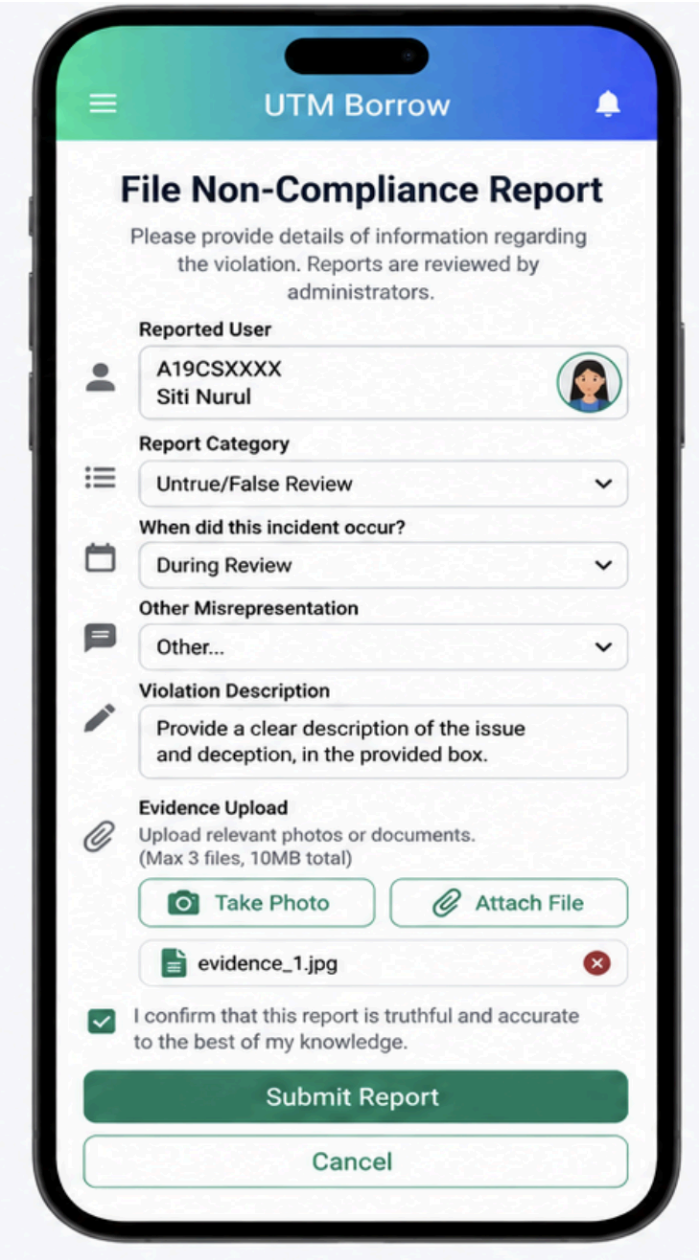



*Figure 3.2.3.4: User Interface for User Reporting page for PROFILE & TRUST SCORE MODULE*

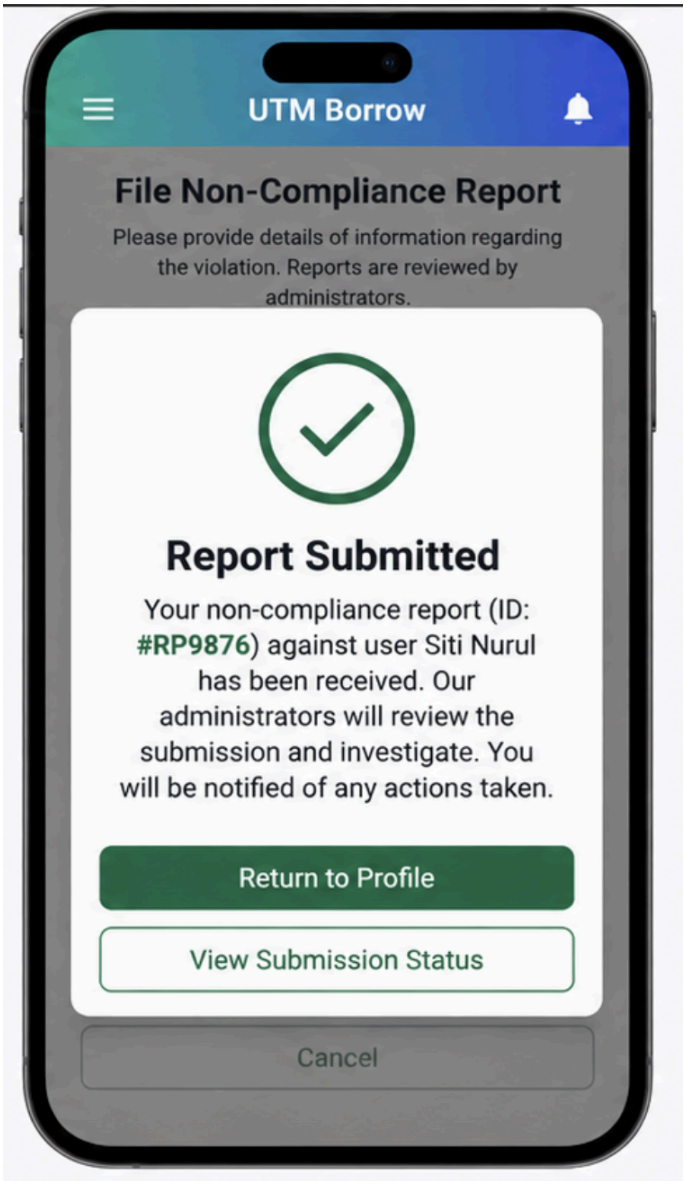



*Figure 3.2.3.5: User Interface for Report Submition page for PROFILE & TRUST SCORE MODULE*

## 3.2. USER ACTIVITY DASHBOARD MODULE

The User Activity Dashboard functions as a data aggregation layer. Rather than establishing novel base entities, it executes complex read queries across the Transaction and Resource Catalog subsystems to render a unified, real-time interface of a user's active lending and borrowing commitments.

## 3.2.1. Data Design

Figure 3.3.1 outlines the cross-module dependencies required for the dashboard. The `Users` table acts as the primary query anchor to filter and join records from the external `Transactions` and `Items` tables, enabling role-specific data retrieval.

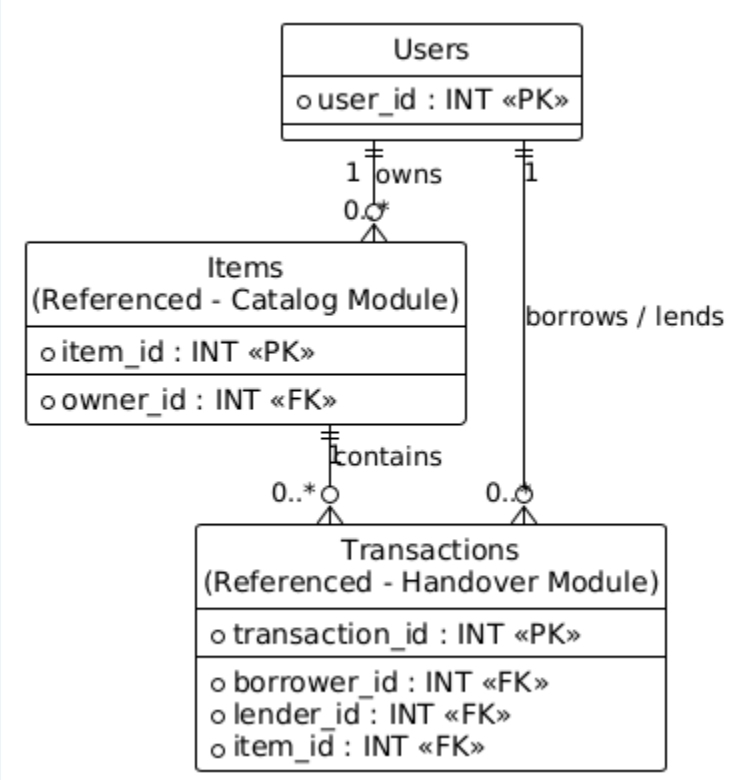



*Figure 3.3.1: Entity Relationship Design (ERD) for USER ACTIVITY DASHBOARD MODULE*

## 3.2.2. Data Dictionary & Description

Because this module acts strictly as a presentation layer, it does not manage proprietary tables. The referenced entities below dictate the specific attributes retrieved from the Handover and Catalog modules necessary to populate the dashboard views.

- **Entity: Users**

| Attribute name | Data Type | Description |
|---|---|---|
| user_id | INT | PRIMARY KEY, AUTO_INCREMENT, Used to query specific dashboard active commitments |

*Table 3.3.2.1: Data Dictionary and Description of Users Table*

- **Entity: Transactions**

| Attribute name | Data Type | Description |
|---|---|---|
| transaction_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| borrower_id | INT | NOT NULL, Used to filter the "Items I am Borrowing" list |
| lender_id | INT | NOT NULL, Used to filter the "Items I am Lending" list |
| status | ENUM | NOT NULL, Displays the current state of the lease on the dashboard |
| Borrow_end_date | DATE | NOT NULL, Used to calculate time remaining and trigger visual Urgent Return alerts |

*Table 3.3.2.2: Data Dictionary and Description of Transaction Table*

- **Entity: Items**

| Attribute name | Data Type | Description |
|---|---|---|
| item_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| title | VARCHAR(150) | NOT NULL, Item name displayed in dashboard lists |

*Table 3.3.2.3: Data Dictionary and Description of Items Table*

### 3.2.3. User Interface Design

The dashboard UI design utilizes conditional rendering to separate active commitments into distinct Lender and Borrower views. The following figures demonstrate the visual hierarchy and time-sensitive alerts (such as overdue return warnings) implemented at the presentation layer.

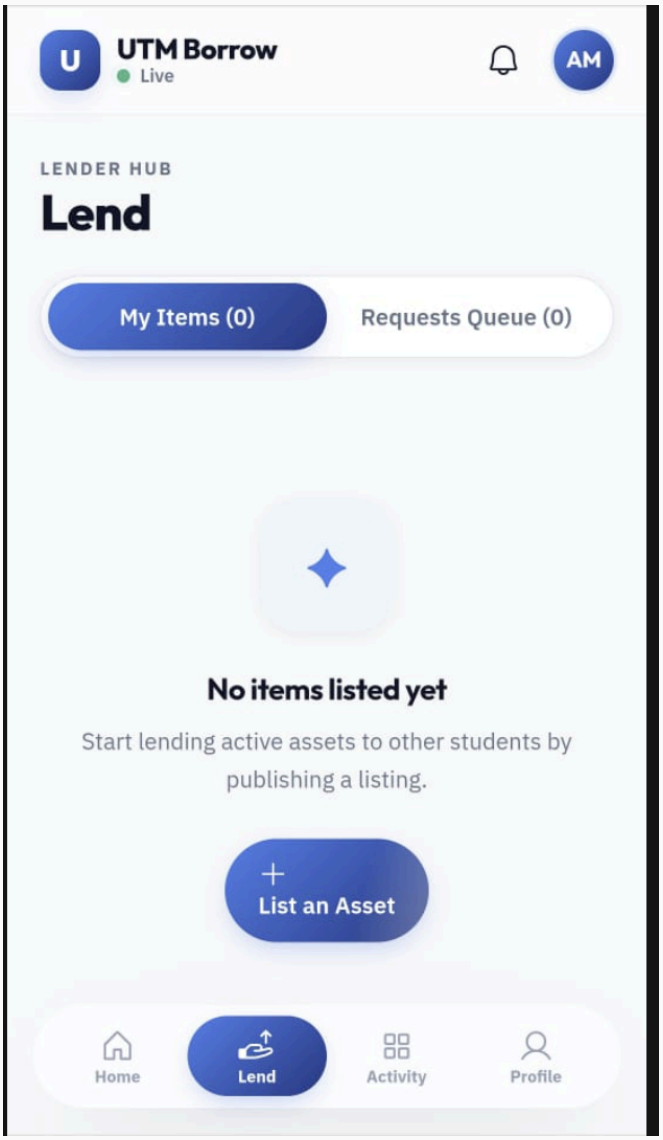



*Figure 3.3.3.1: User Interface for Lending page for USER ACTIVITY DASHBOARD MODULE*

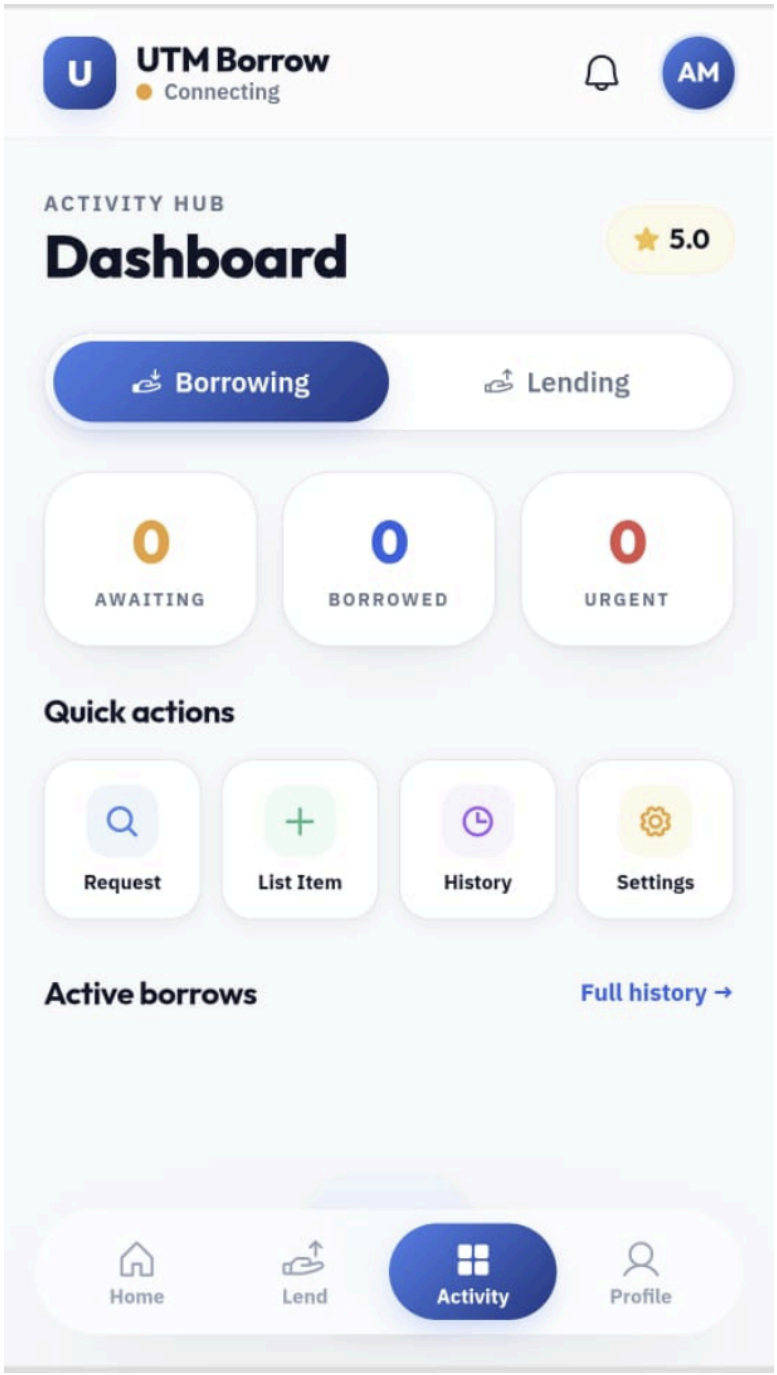



*Figure 3.3.3.3: User Interface for The Dashboard page for USER ACTIVITY DASHBOARD MODULE*

# 4. RESOURCE CATALOG & DISCOVERY SUBSYSTEM

## 4.1. ITEM LISTING & MANAGEMENT MODULE

The Item Listing & Management Module is the core of the Resource Catalog & Discovery Subsystem. It gives item owners full create-read-update-delete control over their listings and is the single source of catalog data that every discovery feature reads from. An owner publishes an item with a title, description, photo, category, condition, and campus location; the listing then becomes visible in the public catalog with an Available status. Owners can edit a listing, toggle it between Public and Private, or remove it, subject to its current transaction state.

### 4.1.1. Data Design

The module is built around a single central Items collection, owned by this subsystem and referenced by the Transaction & Handover and Moderation subsystems. Each item references its owner (a Users record) through owner_id. A secondary Saved_Items collection records the items a user has bookmarked, linking a Users record to an Items record. Figure 4.1.1 shows these relationships.

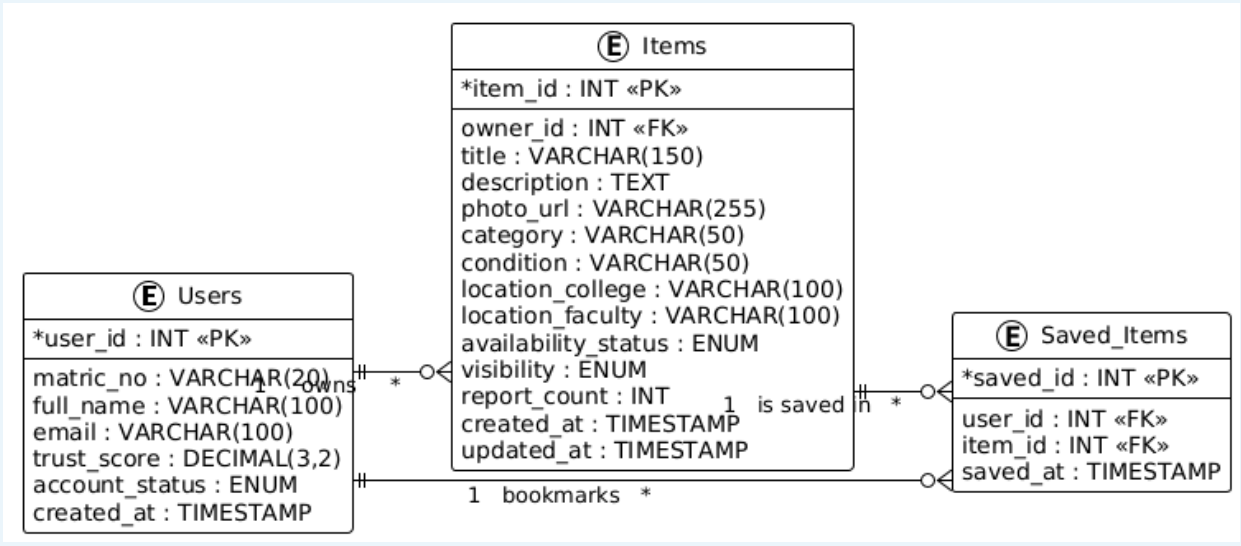



*Figure 4.1.1: Entity Relationship Design (ERD) for ITEM LISTING & MANAGEMENT MODULE*

### 4.1.2. Data Dictionary & Description

The Items collection stores every listing and its lifecycle status. The Saved_Items collection stores user bookmarks. The following tables define their fields and constraints.

- **Entity: Items**

The data dictionary below defines the core catalog entities managed by this module. The Items collection acts as the authoritative source of all resource listings published on the platform, while the Saved_Items collection records user bookmarks for future reference. Together, these entities support listing creation, modification, visibility management, and personalized catalog interaction.

| Attribute name | Data Type | Description |
|---|---|---|
| item_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| owner_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, Identifies the lender who owns this listing |
| title | VARCHAR(150) | NOT NULL, Display name of the item shown in catalog cards |
| description | TEXT | NOT NULL, Detailed item description (max 1000 chars) |
| photo_url | VARCHAR(255) | NULLABLE, URL to the uploaded item image stored in cloud storage |
| category | VARCHAR(50) | NOT NULL, ENUM('Electronics','Textbooks','Lab Equipment','Clothing','Tools','Other') |
| condition | VARCHAR(50) | NOT NULL, ENUM('Like New','Good','Fair','Poor') |
| location_college | VARCHAR(100) | NOT NULL, ENUM('Kolej Tuanku Canselor','Kolej 9','Kolej Perdana','Kolej Rahman Putra','Other') |

| | | |
|---|---|---|
| location_faculty | VARCHAR(100) | NULLABLE, ENUM('Computing','Engineering','Science','Built Environment','Other') |
| availability_status | VARCHAR(20) | NOT NULL, DEFAULT 'Available', ENUM('Available','Pending','Borrowed','Removed') |
| visibility | VARCHAR(20) | NOT NULL, DEFAULT 'Public', ENUM('Public','Private') |
| report_count | INT | NOT NULL, DEFAULT 0, Incremented when users report this item |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

- **Entity: Saved_Items**

| Attribute name | Data Type | Description |
|---|---|---|
| saved_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE CASCADE, The user who bookmarked the item |
| item_id | INT | NOT NULL, FOREIGN KEY → items.item_id, ON DELETE CASCADE, The bookmarked item |
| saved_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

### 4.1.3. User Interface Design

The Item Listing & Management Module provides four primary interface states:

1. Create Listing Form – A multi-field form where owners publish new items. Fields include: title, description, photo upload (drag-and-drop supported), category dropdown, condition dropdown, location college dropdown, optional faculty dropdown, and a visibility toggle (Public/Private). The submit button remains disabled until all required fields are validated.

2. My Listings Dashboard – A grid view displaying all items owned by the current user, organized into two tabs: "Active" (Available/Pending/Borrowed status) and "Hidden" (Private/Removed status). Each listing card shows the item photo, title, status badge (color-coded: green for Available, yellow for Pending, blue for Borrowed, gray for Removed), and action buttons: Edit, Toggle Visibility, Delete (with confirmation dialog).

3. Edit Listing Modal – A modal dialog pre-populated with existing item data. All fields are editable, with the same validation as the create form. Changes reflect immediately in the catalog after saving.

4. Public Catalog View – A searchable, filterable grid of all Public, Available items from active users. Each card displays the item photo, title, category, condition, owner's trust score (as stars), and a "Request to Borrow" button for logged-in users who are not the owner.

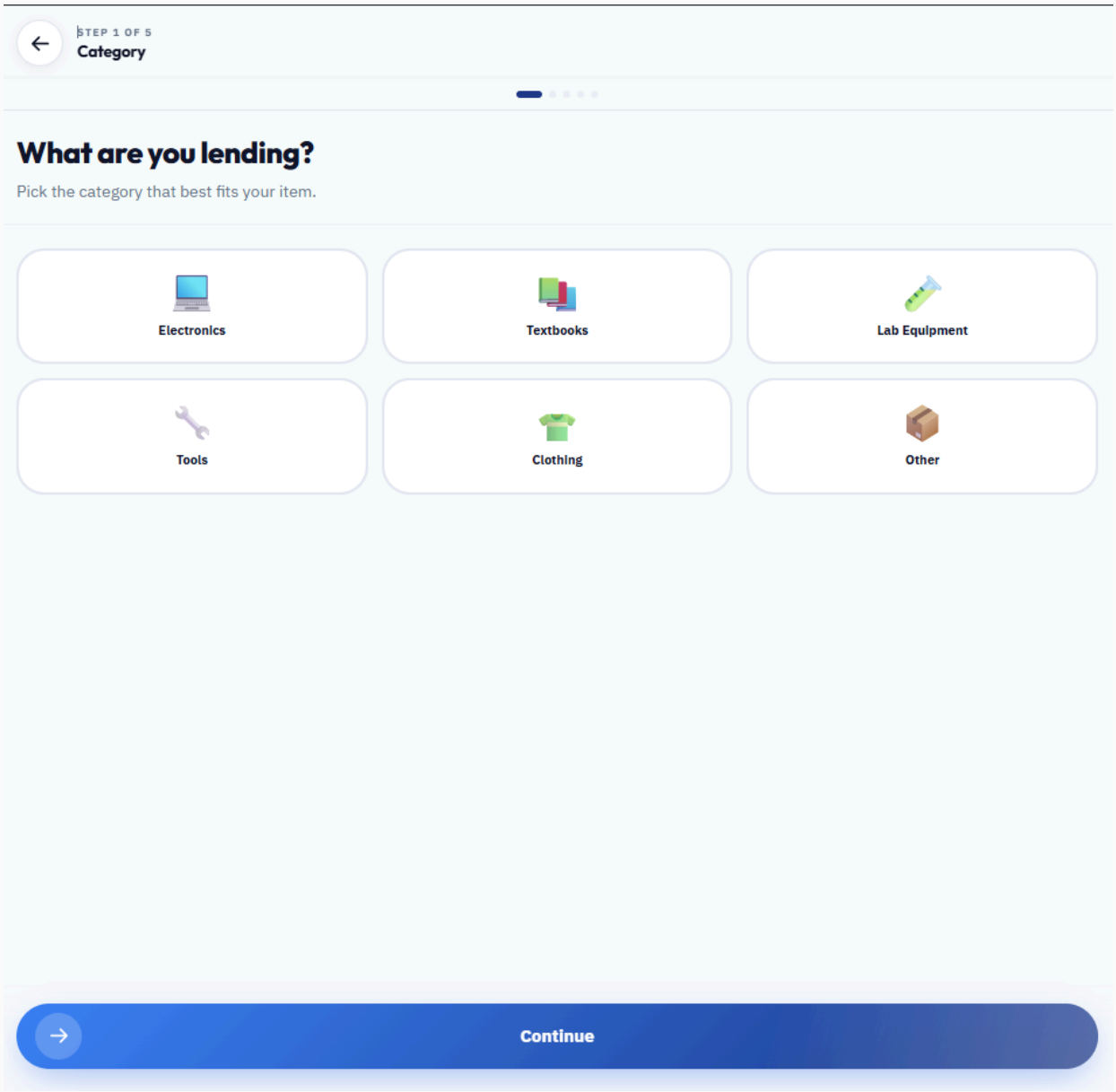



*Figure 4.1.3.1: Add New Item Form for ITEM LISTING & MANAGEMENT MODUL*

## 4.2. CATEGORY & CONDITION METADATA MODULE

The Category & Condition Metadata Module standardizes how items are classified so that discovery and filtering stay consistent. Rather than free-text labels, every listing must select one value from a fixed category list and one value from a fixed condition list. These controlled values are validated on the server for every item write, guaranteeing a clean, filterable catalog.

### 4.2.1. Data Design

This module does not introduce a separate table; it governs two controlled fields on the Items collection — category and condition. Both are validated against fixed server-side value lists on every create and update. Figure 4.2.1 shows these metadata fields as attributes of the Items entity.

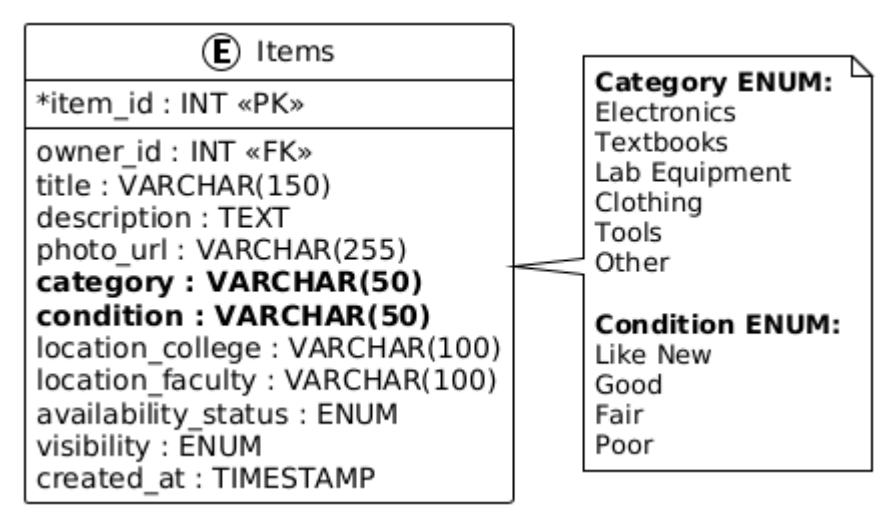



*Figure 4.2.1: Entity Relationship Design (ERD) for CATEGORY & CONDITION METADATA MODULE*

## 4.2.2. Data Dictionary & Description

The two metadata fields below are part of the Items collection and are constrained to fixed value lists enforced by the server.

- **Entity: Items (Category and Condition Fields Only)**

  This module governs the controlled metadata attributes used throughout the catalog. By enforcing predefined category and condition values, the system prevents inconsistent labeling and ensures reliable search and filtering behavior across all catalog listings.

| Attribute name | Data Type | Description |
|---|---|---|
| category | VARCHAR(50) | NOT NULL, ENUM('Electronics','Textbooks','Lab Equipment','Clothing','Tools','Other'). Validated |

| | | |
|---|---|---|
| | | server-side; any request with a value outside this ENUM returns HTTP 422. |
| condition | VARCHAR(50) | NOT NULL, ENUM('Like New','Good','Fair','Poor'). 'Like New' indicates no visible wear; 'Poor' indicates functional but significant damage. |

- **Control Value List (Server-Enforced)**

| Category Options | Condition Options |
|---|---|
| Electronics | Like New |
| Textbooks | Good |
| Lab Equipments | Fair |
| Tools | Poor |
| Clothing | |
| Other | |

## 4.2.3. User Interface Design

The Category & Condition Metadata Module is embedded within the listing form rather than as a standalone interface:

1. Category Selector – A dropdown menu rendered as a <select> element displaying all six category options. The default empty state shows "-- Select Category --". On selection, the category is sent to the backend as a validated string.

2. Condition Selector – A row of four labeled radio buttons or a segmented picker (Like New, Good, Fair, Poor) with descriptive tooltips explaining each condition level. This design reduces selection errors compared to a dropdown.

3. Form Validation Feedback – If the user attempts to submit without selecting both category and condition, inline error messages appear beneath each field: "Category is required" and "Condition is required". The submit button remains disabled until both selections are made.

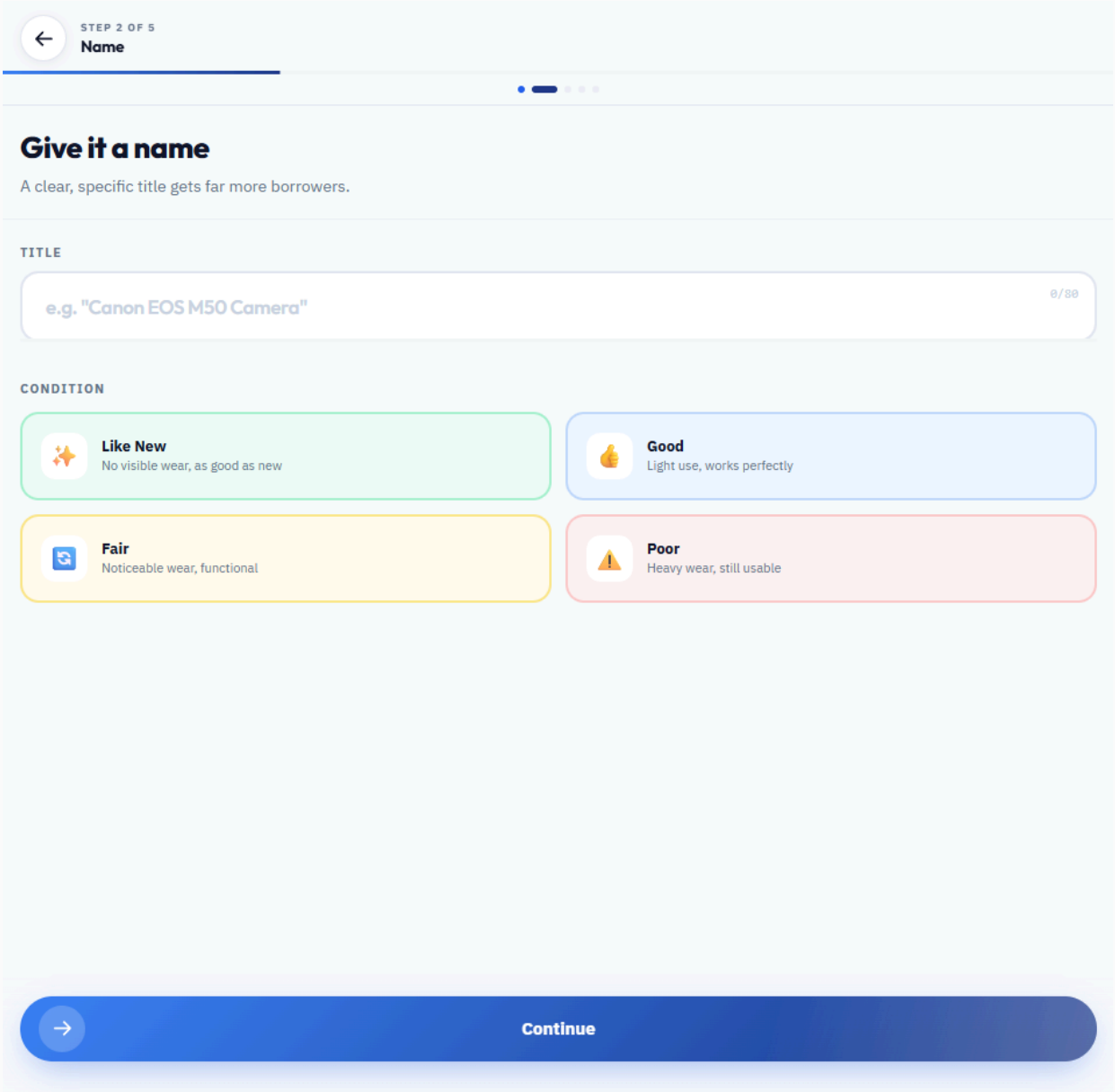
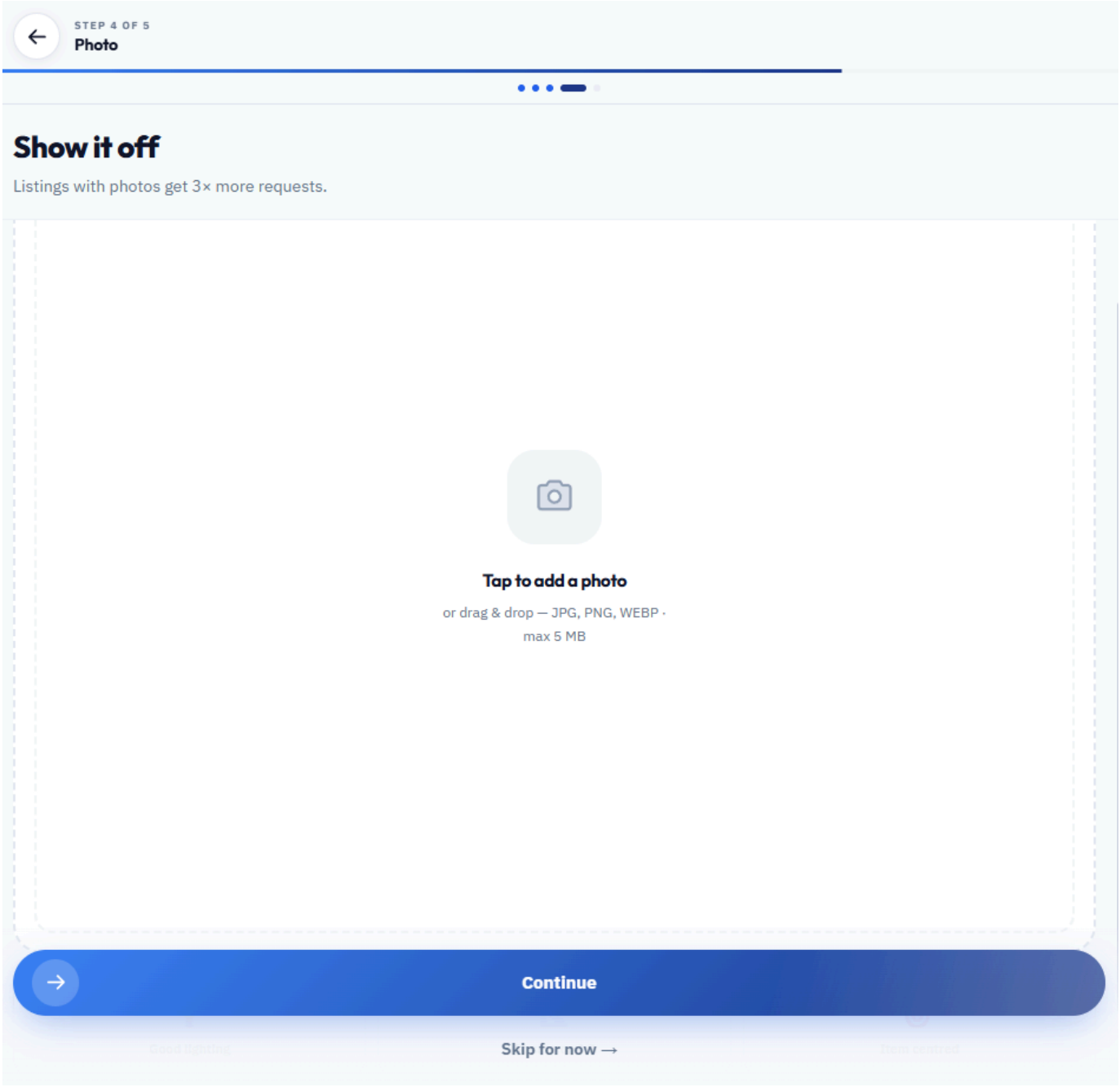



*Figure 4.2.3.1: Browse by Category for CATEGORY & CONDITION METADATA MODUL*




## 4.3. LOCATION-BASED FILTER MODULE

The Location-Based Filter Module helps borrowers find items close to them on campus without any GPS tracking. Each listing records the owner's residential college and (optionally) faculty, chosen from fixed UTM location lists. Borrowers filter the catalog by these locations — alongside category, condition, and free-text search — to surface nearby resources and reduce travel.

### 4.3.1. Data Design

Like the metadata module, this module governs fields on the Items collection rather than a separate table: location_college and location_faculty. Both are validated against fixed UTM location lists. The catalog browse query filters items by these fields (combined with category, condition, and a text

query) and returns only Available, Public items owned by active users. Figure 4.3.1 shows these location attributes on the Items entity.

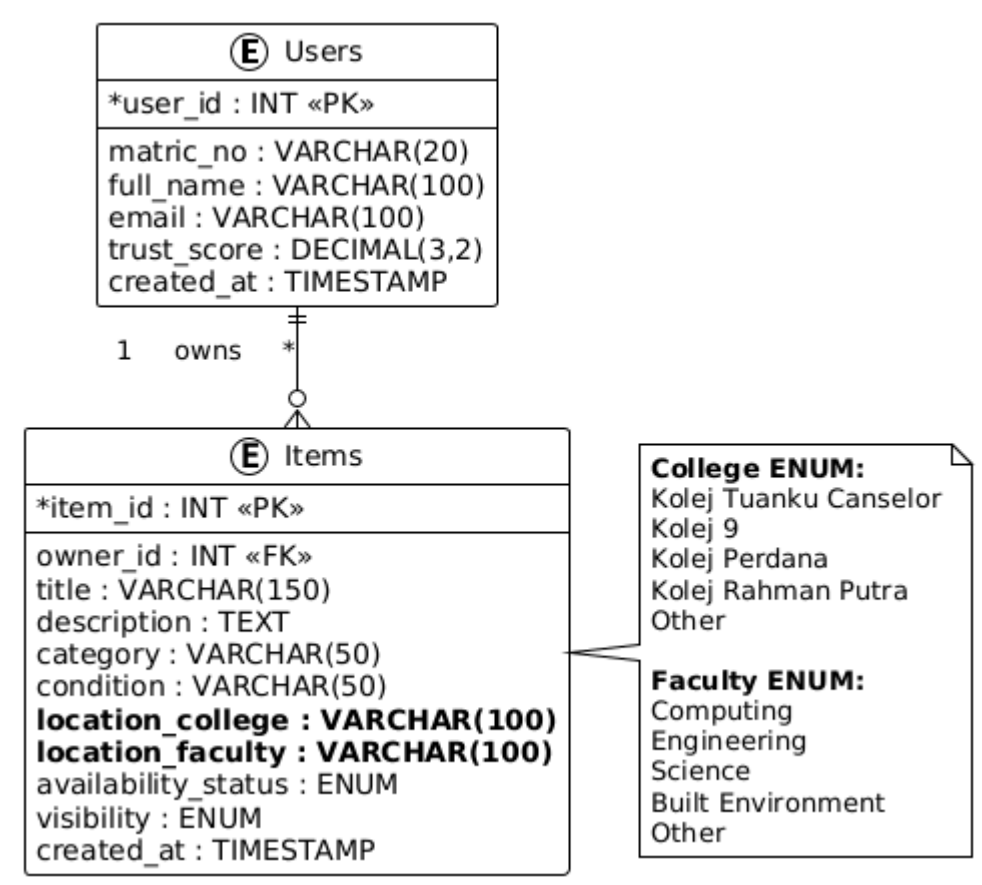



*Figure 4.3.1: Entity Relationship Design (ERD) for LOCATION-BASED FILTER MODULE*

## 4.3.2. Data Dictionary & Description

The two location fields below are part of the Items collection and are constrained to fixed UTM value lists.

- **Entity: Items (Location Fields Only)**

The location attributes below support campus-aware resource discovery without requiring GPS tracking. By associating listings with validated UTM residential colleges and faculties, the

platform enables borrowers to identify nearby resources and minimize travel during item handover and return.

| Attribute name | Data Type | Description |
|---|---|---|
| location_college | VARCHAR(100) | NOT NULL, ENUM('Kolej Tuanku Canselor','Kolej 9','Kolej Perdana','Kolej Rahman Putra','Other'). The residential college where the item is physically located. |
| location_faculty | VARCHAR(100) | NULLABLE, ENUM('Computing','Engineering','Science','Built Environment','Other'). Optional academic building context for daytime pickups. |

- **Controlled Value List (UTM Location Data):**

| College Options | Faculty Options (Optional) |
|---|---|
| Kolej Tuanku Canselor | Computing |
| Kolej 9 | Engineering |
| Kolej Perdana | Science |
| Kolej Rahman Putra | Built Environment |
| Other | Other |

## 4.3.3. User Interface Design

The Location-Based Filter Module provides filtering controls on the catalog page and stores location data on each listing:

1. Filter Sidebar / Top Bar – Located above the catalog grid, containing:

- College filter: Multi-select dropdown or checkbox group listing all college options.
- Faculty filter: Optional dropdown (disabled by default, enabled when user opts to filter by faculty).
- Clear Filters button: Resets all location filters to default.

2. Combined Filter Logic – The catalog query automatically combines location filters with category, condition, and text search. For example, selecting "Kolej 9" returns only items whose location_college matches.

3. Location Display on Listing Cards – Each item card shows a small location badge (e.g., "📍 KTC") that appears on hover or beneath the condition tag, helping borrowers quickly identify nearby items without opening the detail view.

4. Zero-Results State – When location filters return no items, a friendly empty state appears: "No items found at this college. Try removing filters or checking nearby colleges."

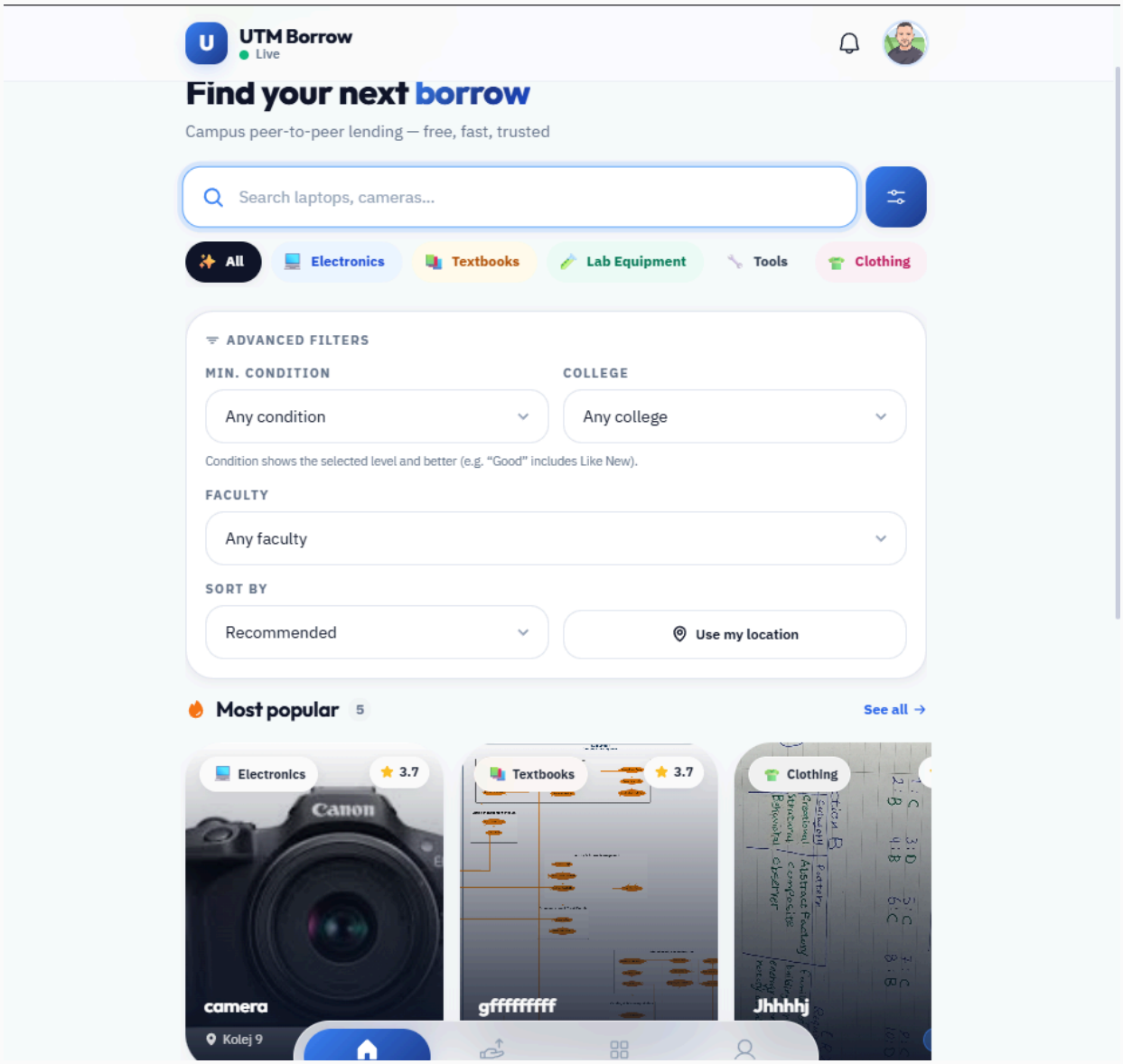
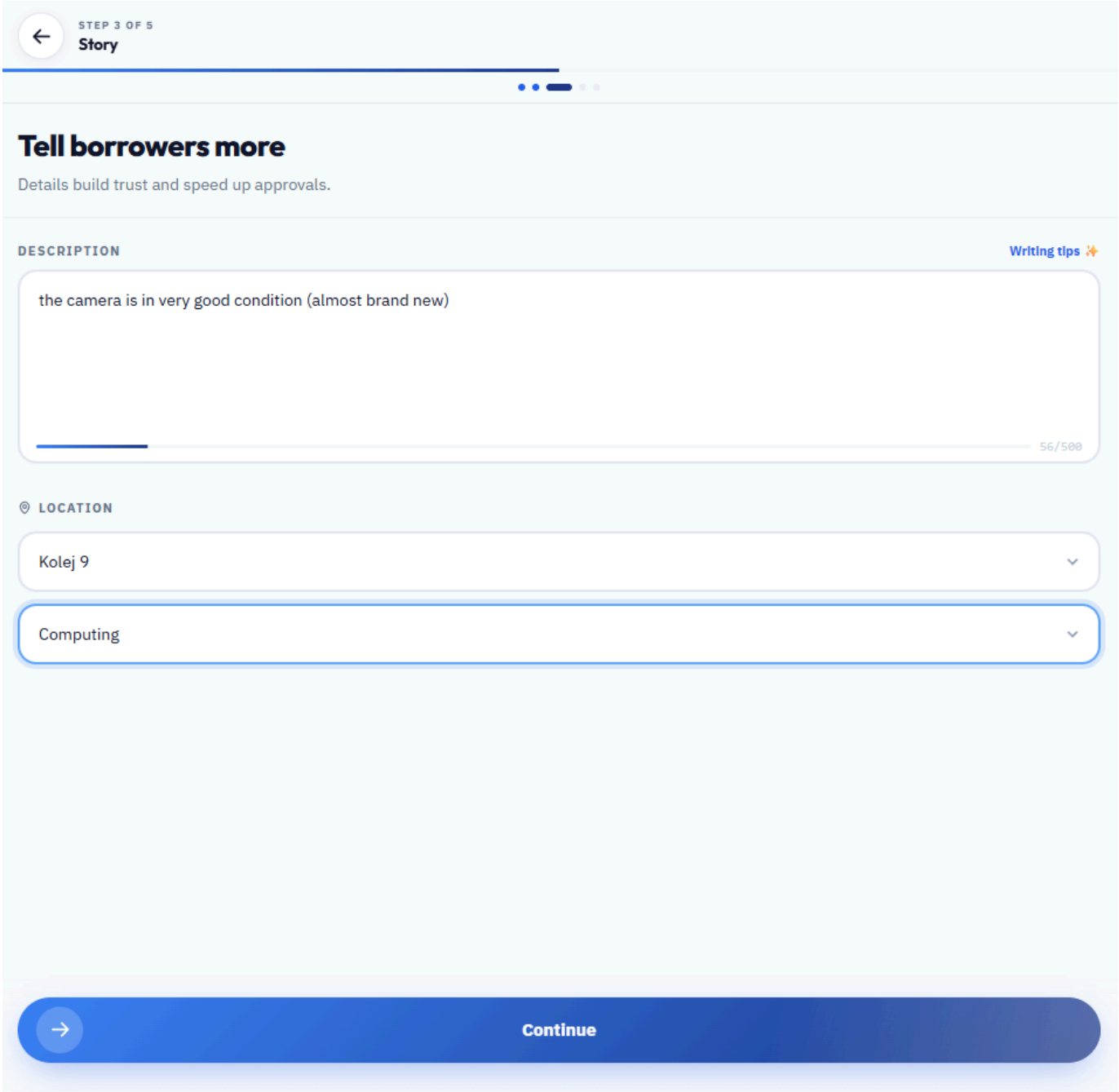



*Figure 4.3.3.1: Location Filter Search for LOCATION-BASED FILTER MODULE*


STEP 3 OF 5
Story
Tell borrowers more
Details build trust and speed up approvals.
DESCRIPTION
Writing tips
the camera is in very good condition (almost brand new)
56/500
LOCATION
Kolej 9
Computing
Continue

# 5. TRANSACTION & HANDOVER SUBSYSTEM- Alsakkaf

## 5.1. REQUEST & APPROVAL WORKFLOW MODULE- Alsakkaf

This module manages the digital agreement phase of a loan. A borrower submits a dated request for an available item; the system validates the dates and atomically locks the item from Available to Pending so no two students can request it at once. The lender then approves or rejects the request, and either party may cancel before the physical handover occurs. Every state change is recorded.

### 5.1.1. Data Design

The module centers on the Transactions collection, which references Users (borrower and lender) and Items. Each state change writes an immutable Transaction_State_Logs record, and every transition notifies the affected party through the Notifications collection. Figure 5.1.1 shows these relationships.

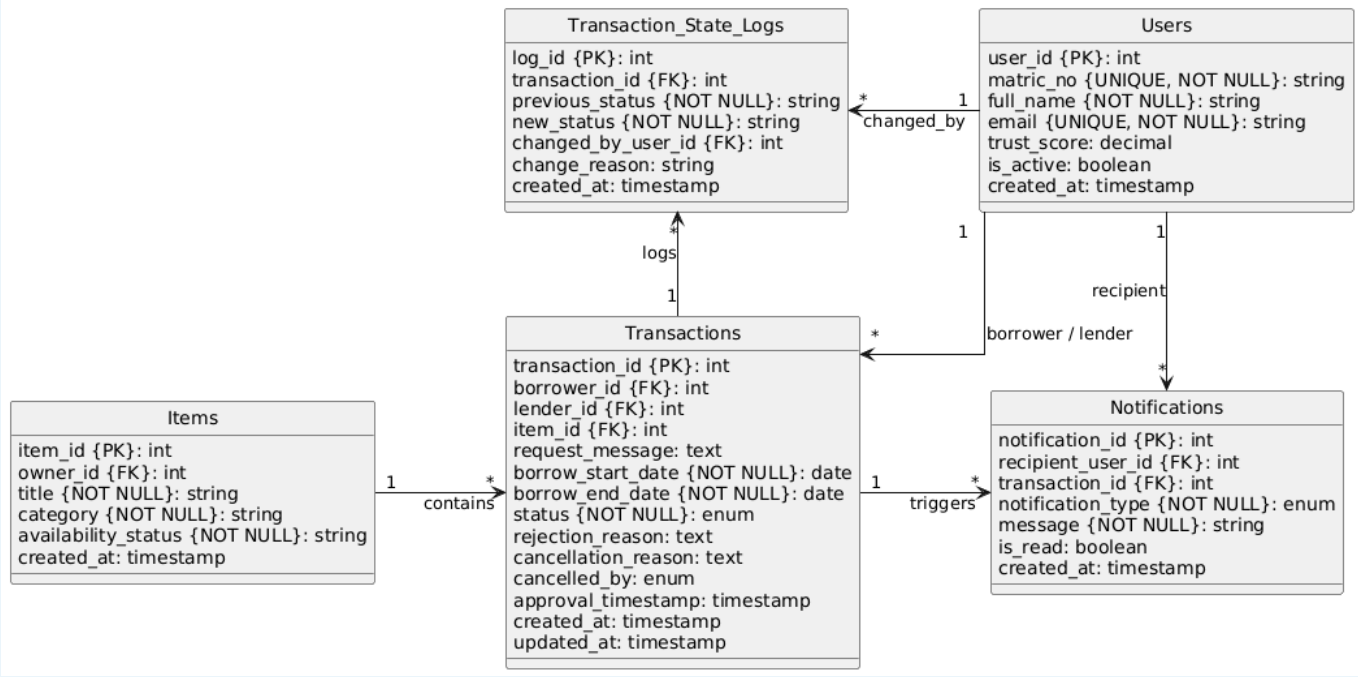



*Figure 5.1.1: Entity Relationship Design (ERD) for REQUEST & APPROVAL WORKFLOW MODULE*

## 5.1.2. Data Dictionary & Description

The tables below define the entities this module reads and writes. `Users` and `Items` are referenced from other subsystems; `Transactions`, `Transaction_State_Logs`, and `Notifications` are detailed here.

- **Entity: Users** **(Referenced — owned by Identity & User Management Subsystem)**

The borrower and lender of a transaction; the lender reviews the borrower's Trust Score before approving.

| Attribute name | Data Type | Description |
|---|---|---|
| user_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| matric_no | VARCHAR(20) | NOT NULL, UNIQUE, UTM student matriculation number |
| full_name | VARCHAR(100) | NOT NULL, Display name shown to counterparties |
| email | VARCHAR(100) | NOT NULL, UNIQUE, UTM institutional email |
| trust_score | DECIMAL(3,2) | DEFAULT 5.00, Range 0.00–5.00, Used by lender during approval review |
| account_status | ENUM | NOT NULL, DEFAULT 'Active', Options: 'Active','Suspended','Banned' |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

*Table 5.1.2.1: Data Dictionary and Description of Users Table*

- **Entity: Items** **(Referenced — owned by Identity & User Management Subsystem)**

The item being requested; this module locks its availability_status from Available to Pending on submission.

| Attribute name | Data Type | Description |
|---|---|---|
| item_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| owner_id | INT | NOT NULL, FOREIGN KEY → users.user_id, Identifies the lender |
| title | VARCHAR(150) | NOT NULL, Item display name |
| category | VARCHAR(50) | NOT NULL, Item classification (e.g., Electronics, Books, Tools) |
| availability_status | VARCHAR(20) | NOT NULL, DEFAULT 'Available', ENUM('Available','Pending','Borrowed','Removed'), Locked by this module on request submission |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

*Table 5.1.2.2: Data Dictionary and Description of Items Table*

- **Entity: Transactions** **(Referenced — owned by Identity & User Management Subsystem)**

The central record of a borrow agreement and its status across the full lifecycle.

| Attribute name | Data Type | Description |
|---|---|---|
| transaction_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| borrower_id | INT | NOT NULL, FOREIGN KEY → users.user_id, The student submitting the request |
| lender_id | INT | NOT NULL, FOREIGN KEY → users.user_id, The item owner reviewing the request |
| item_id | INT | NOT NULL, FOREIGN KEY → items.item_id, The requested resource |
| request_message | TEXT | NULLABLE, Optional contextual note from borrower (max 500 chars enforced at app layer) |
| borrow_start_date | DATE | NOT NULL, Must be ≥ CURRENT_DATE (validated at submission) |
| borrow_end_date | DATE | NOT NULL, Must be > borrow_start_date (validated at submission) |
| status | ENUM | NOT NULL, DEFAULT 'Pending', Options: 'Pending', 'Approved', 'Rejected', 'Cancelled' |
| rejection_reason | TEXT | NULLABLE, Populated only when status = 'Rejected' |
| cancellation_reason | TEXT | NULLABLE, Populated only when status = 'Cancelled' |
| cancelled_by | ENUM | NULLABLE, Options: 'Borrower', 'Lender', Required when status = 'Cancelled' |
| approval_timestamp | TIMESTAMP | NULLABLE, Set exactly once when status transitions to 'Approved' |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, Original request submission time |
| updated_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP |

*Table 5.1.2.3: Data Dictionary and Description of Transactions Table*

- **Entity: Transaction_State_Logs**

An append-only audit trail; one row per state transition, recording who changed it and when.

| Attribute name | Data Type | Description |
|---|---|---|
| log_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| transaction_id | INT | NOT NULL, FOREIGN KEY → transactions.transaction_id, ON DELETE CASCADE |
| previous_status | VARCHAR(20) | NOT NULL, The status value before the transition |
| new_status | VARCHAR(20) | NOT NULL, The status value after the transition |
| changed_by_user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, Identifies the actor who triggered the transition |
| change_reason | VARCHAR(255) | NULLABLE, Optional human-readable explanation for the transition |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, NOT NULL, Immutable timestamp of state change |

*Table 5.1.2.4: Data Dictionary and Description of Transaction_State_Logs Table*

- **Entity: Notifications**

Real-time alerts dispatched to a recipient on each relevant transaction event.

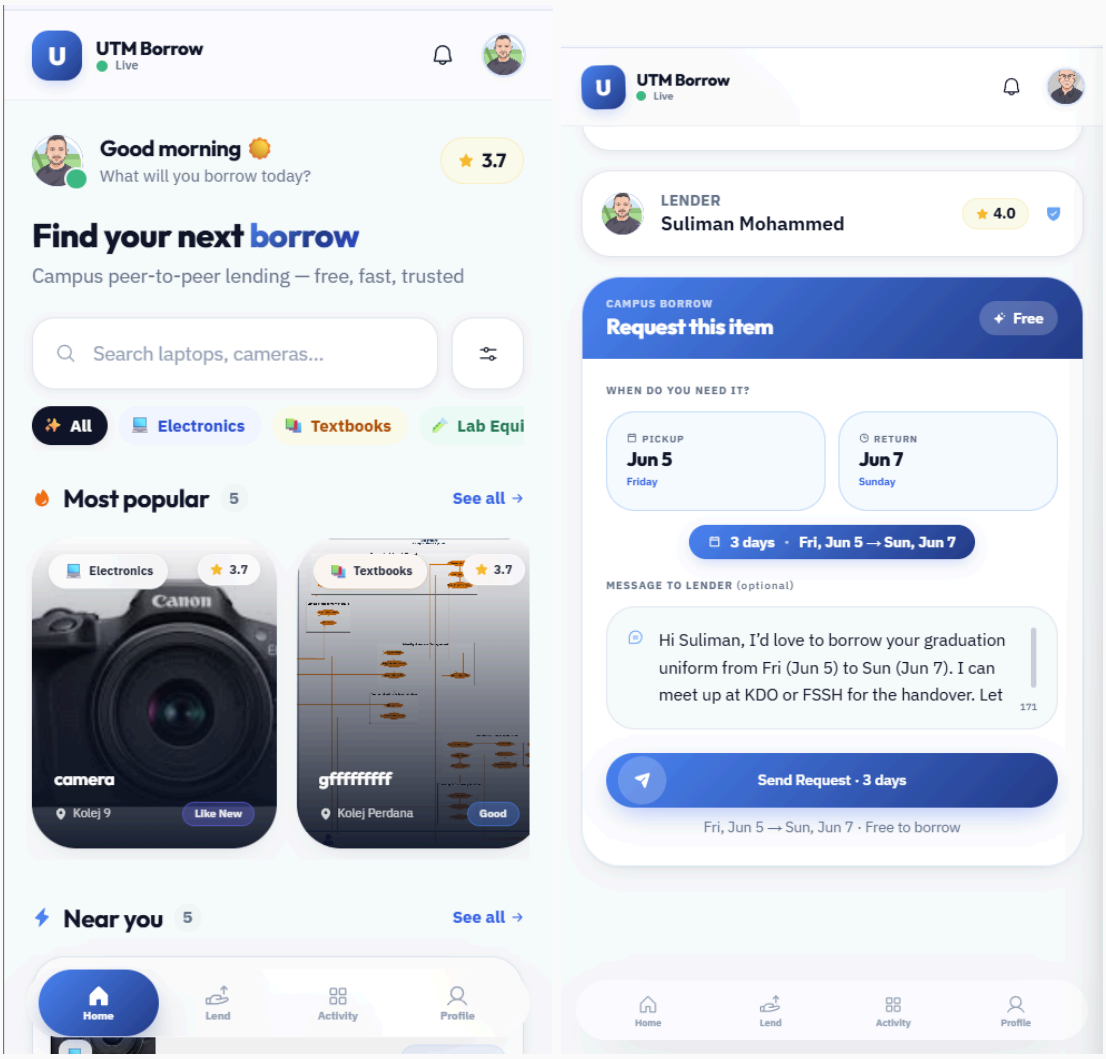
| Attribute name | Data Type | Description |
|---|---|---|
| notification_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| recipient_user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE CASCADE, The user receiving the alert |
| transaction_id | INT | NOT NULL, FOREIGN KEY → transactions.transaction_id, ON DELETE CASCADE, Contextual transaction reference |
| notification_type | ENUM | NOT NULL, Options: 'RequestReceived', 'RequestApproved', 'RequestRejected', 'RequestCancelled' |
| message | VARCHAR(255) | NOT NULL, Pre-formatted human-readable alert text |
| is_read | BOOLEAN | DEFAULT FALSE, Toggled to TRUE when user opens the notification |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, Dispatch time |

*Table 5.1.2.5: Data Dictionary and Description of Notifications Table*

### 5.1.3. User Interface Design

**Borrower Submits a Request**

This interface displays the detailed view of a specific resource item, including its current availability status, category, condition, and owner's trust score. It features date selection fields for the borrowing period, a text field for optional contextual notes, and the primary "Request This Item" button which atomically changes the item's state to Pending upon submission.


UTM Borrow
Live
Good morning
What will you borrow today?
3.7
Find your next borrow
Campus peer-to-peer lending — free, fast, trusted
Search laptops, cameras...
All
Electronics
Textbooks
Lab Equi
Most popular 5
See all →
Electronics
3.7
Canon
camera
Kolej 9
Like New
Textbooks
3.7
gfffffffff
Kolej Perdana
Good
Near you 5
See all →
Home
Lend
Activity
Profile
UTM Borrow
Live
LENDER
Suliman Mohammed
4.0
CAMPUS BORROW
Request this item
Free
WHEN DO YOU NEED IT?
PICKUP
Jun 5
Friday
RETURN
Jun 7
Sunday
3 days · Fri, Jun 5 → Sun, Jun 7
MESSAGE TO LENDER (optional)
Hi Suliman, I'd love to borrow your graduation uniform from Fri (Jun 5) to Sun (Jun 7). I can meet up at KDO or FSSH for the handover. Let
171
Send Request · 3 days
Fri, Jun 5 → Sun, Jun 7 · Free to borrow
Home
Lend
Activity
Profile

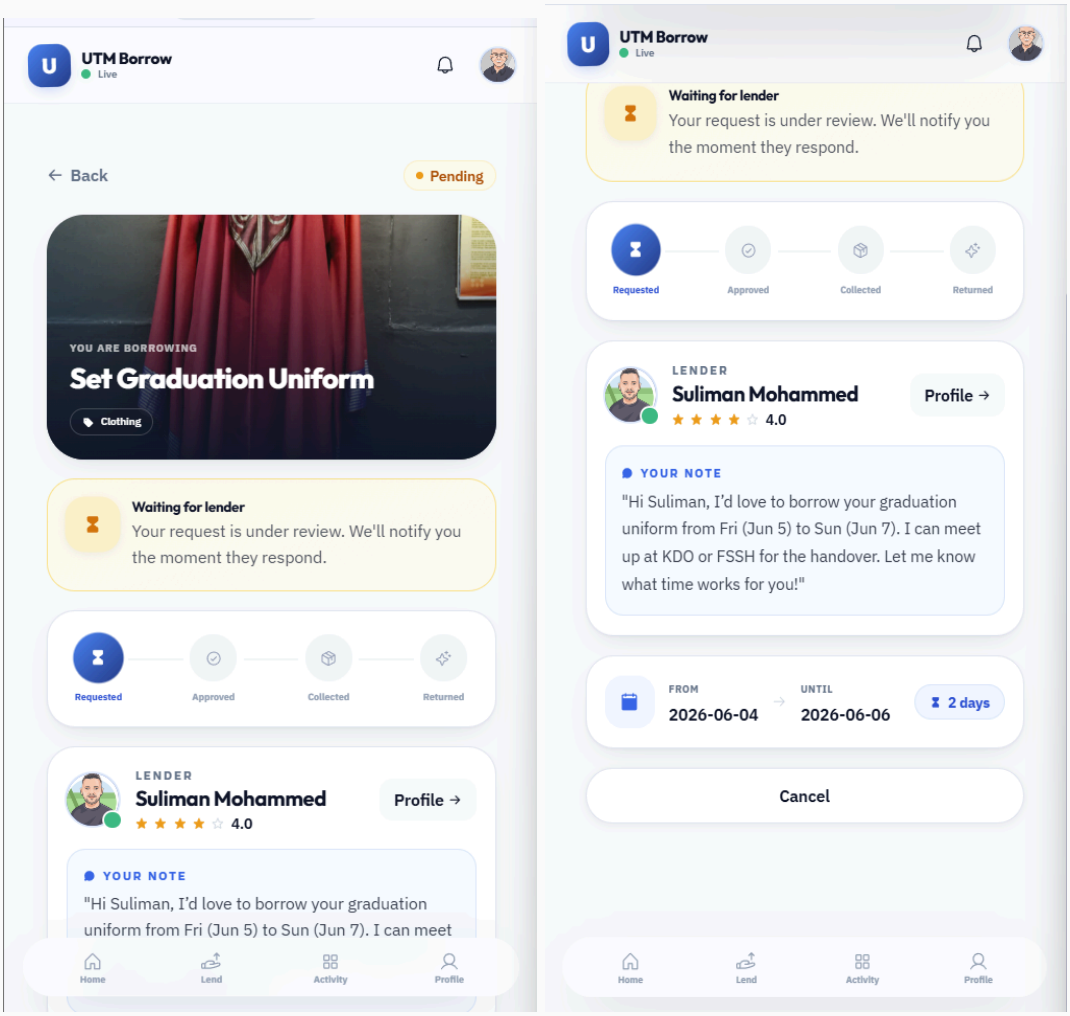



*Figure 5.1.3.1: Item Detail and Request Submission Screens*

**Lender Decides on Request**

This interface displays the lender's incoming requests dashboard, allowing the resource owner to evaluate the borrower's identity, matric number, and dynamic Trust Score. It provides explicit action gates to approve or reject the transactional request, alongside a mandatory text input for rejection rationale if applicable.

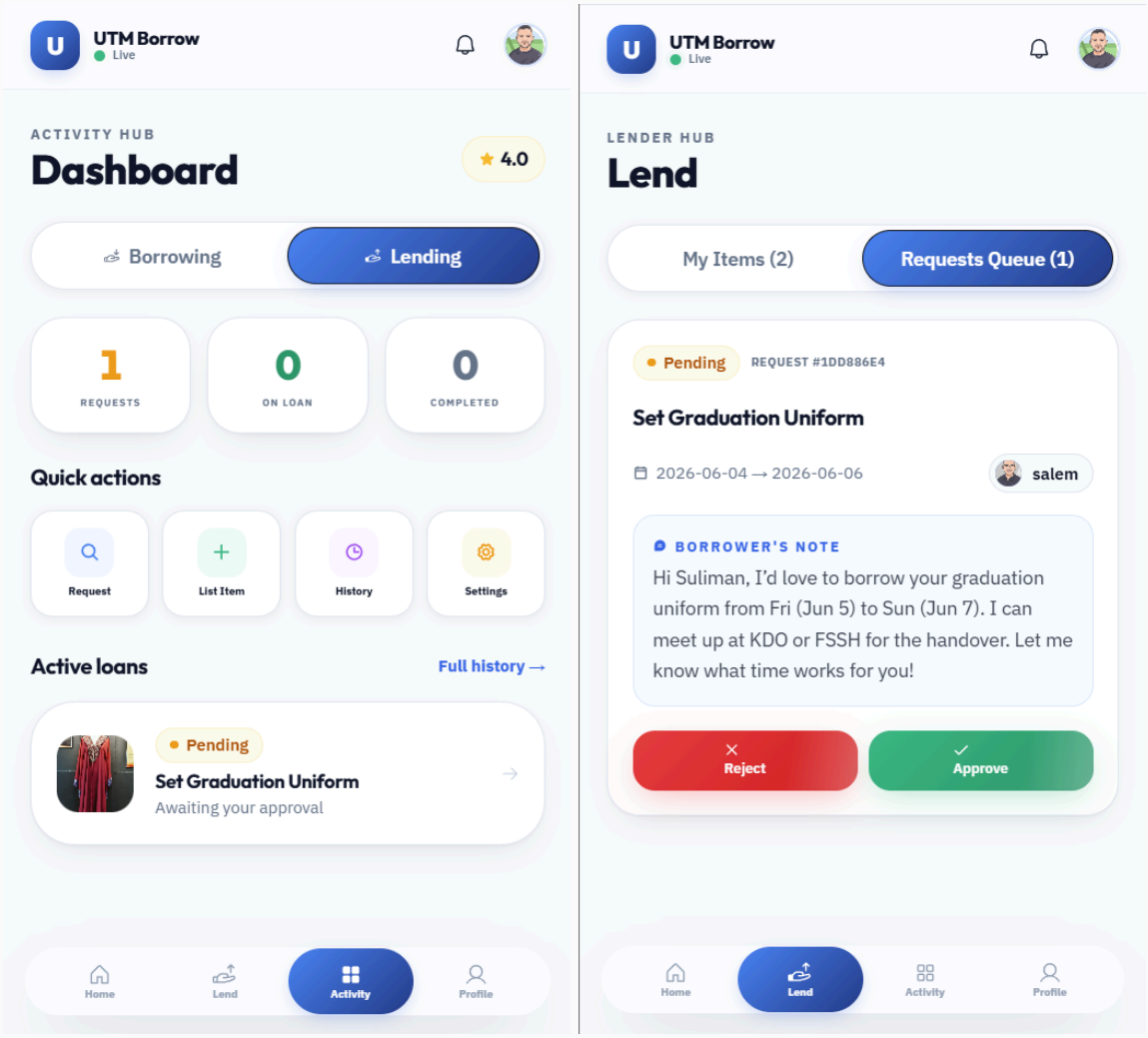



*Figure 5.1.3.1: Lender Pending Request Review Screen*

## 5.2. QR VERIFICATION MODULE- Alsakkaf

This module produces and validates the cryptographic proof of physical exchange. When a request is approved, the borrower can display an HMAC-SHA256 signed, time-limited QR code; the lender scans it to confirm handover (Approved → Borrowed) and scans again at the end to confirm return (Borrowed → Completed). Every scan attempt is logged with its precise result.

### 5.2.1. Data Design

The module adds three collections — QR_Tokens (one signed token per transaction), Scan_Events (every scan attempt and outcome), and Lease_Cycles (one lease lifecycle per transaction) — all referencing Transactions. Figure 5.2.1 shows these relationships.

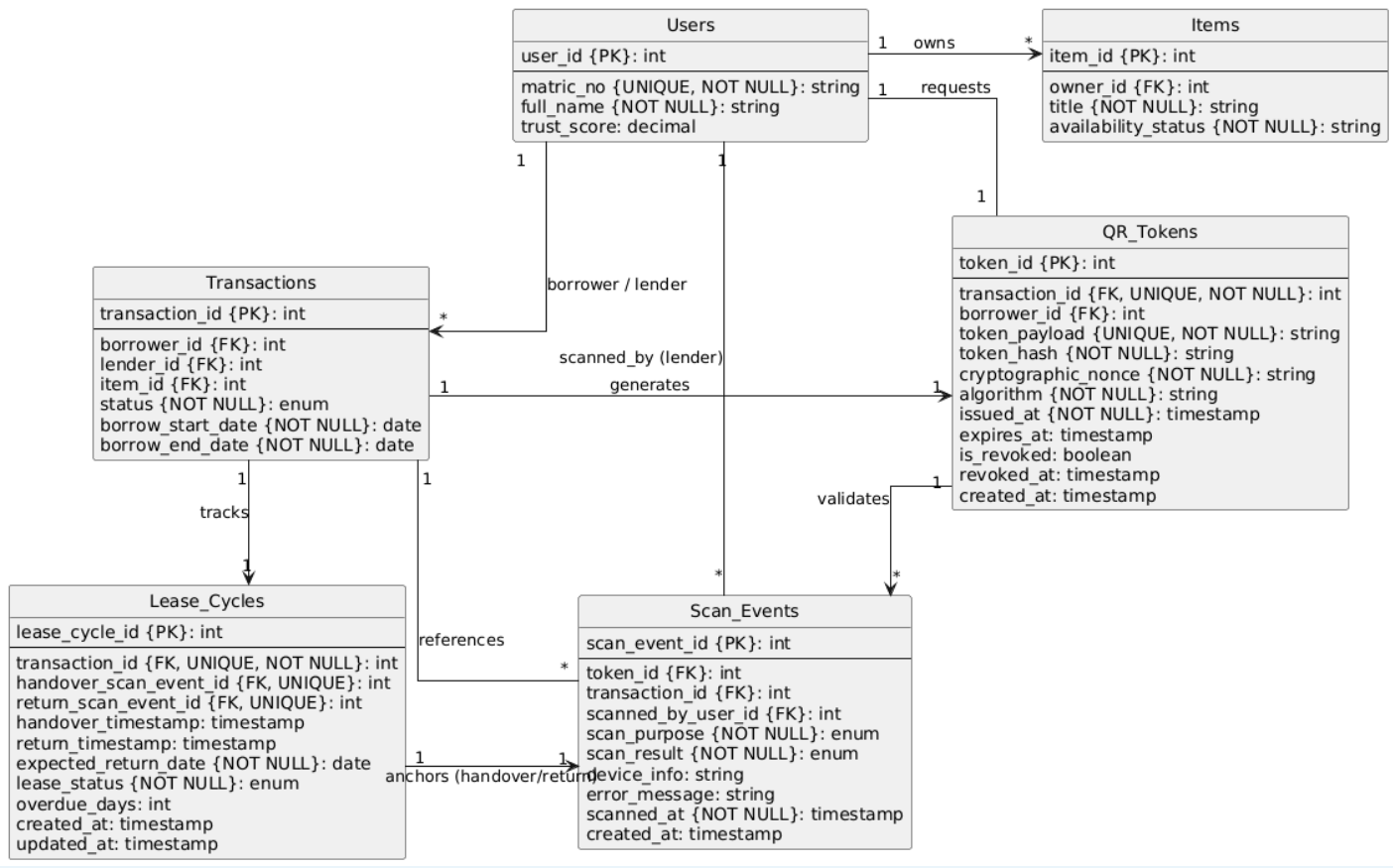



*Figure 5.2.1: Entity Relationship Design (ERD) for QR VERIFICATION MODULE*

### 5.2.2. Data Dictionary & Description

The tables below define the QR entities and the referenced Users, Items, and Transactions fields relevant to verification.

- **Entity: Users** **(Referenced — owned by Identity & User Management Subsystem)**

Identifies the scanner (always the lender) and the token-bound borrower.

| Attribute name | Data Type | Description |
|---|---|---|
| user_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| matric_no | VARCHAR(20) | NOT NULL, UNIQUE, UTM matriculation number |
| full_name | VARCHAR(100) | NOT NULL, Display name for scan attribution |
| trust_score | DECIMAL(3,2) | DEFAULT 5.00, Range 0.00–5.00, Updated after lease completion |

*Table 5.2.2.1: Data Dictionary and Description of Notifications Table*

- **Entity: Items** **(Referenced — owned by Identity & User Management Subsystem)**

Its availability_status flips to Borrowed on handover and back to Available on return.

| Attribute name | Data Type | Description |
|---|---|---|
| item_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| owner_id | INT | NOT NULL, FOREIGN KEY → users.user_id, Identifies the lender |
| title | VARCHAR(150) | NOT NULL, Item display name |
| availability_status | VARCHAR(20) | NOT NULL, DEFAULT 'Available', Transitions to 'Borrowed' on handover scan and back to 'Available' on return scan |

*Table 5.2.2.2: Data Dictionary and Description of Notifications Table*

- **Entity: Transactions** **(Referenced — owned by Identity & User Management Subsystem)**

Token issuance requires Approved; scans drive the Approved → Borrowed → Completed transitions.

| Attribute name | Data Type | Description |
|---|---|---|
| transaction_id | INT | PRIMARY KEY, AUTO_INCREMENT |

| | | |
|---|---|---|
| borrower_id | INT | NOT NULL, FOREIGN KEY → users.user_id, Token recipient |
| lender_id | INT | NOT NULL, FOREIGN KEY → users.user_id, Token scanner |
| item_id | INT | NOT NULL, FOREIGN KEY → items.item_id, The leased resource |
| status | ENUM | NOT NULL, Options: 'Pending', 'Approved', 'Rejected', 'Cancelled', 'Borrowed', 'Completed'. Token issuance requires status='Approved'; handover scan transitions Approved → Borrowed; return scan transitions Borrowed → Completed |
| borrow_start_date | DATE | NOT NULL, Original requested start date |
| borrow_end_date | DATE | NOT NULL, Original requested end date |

***Table 5.2.2.3: Data Dictionary and Description of Users Table***

- **Entity: QR_Tokens**

The signed token rendered into the QR image, one per transaction.

| Attribute name | Data Type | Description |
|---|---|---|
| token_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| transaction_id | INT | NOT NULL, UNIQUE, FOREIGN KEY → transactions.transaction_id, ON DELETE CASCADE, Enforces strict one-token-per-transaction policy |
| borrower_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, Token-bound borrower identity |
| token_payload | VARCHAR(512) | NOT NULL, UNIQUE, Base64-encoded JWT containing {transaction_id, borrower_id, nonce, iat} — this is the raw data rendered into the QR image |
| token_hash | VARCHAR(255) | NOT NULL, HMAC-SHA256 cryptographic signature computed server-side using a private secret key — validated on every scan |
| cryptographic_nonce | VARCHAR(64) | NOT NULL, Cryptographically random 32-byte salt rendered as 64-char hex string — guarantees uniqueness even for identical transaction parameters |
| algorithm | VARCHAR(20) | NOT NULL, DEFAULT 'HMAC-SHA256', Signing algorithm identifier for future cryptographic upgrades |
| issued_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP, Token generation moment |
| expires_at | TIMESTAMP | NULLABLE, NULL indicates the token remains valid for the entire lease lifecycle |
| is_revoked | BOOLEAN | NOT NULL, DEFAULT FALSE, TRUE prevents all future scan validation |
| revoked_at | TIMESTAMP | NULLABLE, Set when is_revoked transitions to TRUE |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, NOT NULL |

*Table 5.2.2.4: Data Dictionary and Description of Items Table*

- **Entity: Scan_Events**

A forensic log of every scan, successful or failed.

| Attribute name | Data Type | Description |
|---|---|---|
| scan_event_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| token_id | INT | NOT NULL, FOREIGN KEY → qr_tokens.token_id, ON DELETE CASCADE, The token presented during the scan attempt |
| transaction_id | INT | NOT NULL, FOREIGN KEY → transactions.transaction_id, ON DELETE CASCADE, Denormalized for efficient transaction-history queries |
| scanned_by_user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, Always references the lender's user_id |
| scan_purpose | ENUM | NOT NULL, Options: 'Handover', 'Return', Declared by the lender via UI tab selection before scanning |
| scan_result | ENUM | NOT NULL, Options: 'Success', 'Invalid_Token', 'Wrong_Transaction', 'Already_Used', 'Expired', 'State_Mismatch', 'Camera_Error', Determined by server-side validation |
| device_info | VARCHAR(255) | NULLABLE, Optional device fingerprint string (OS, model, app version) captured for forensic context |
| error_message | VARCHAR(255) | NULLABLE, Populated only when scan_result ≠ 'Success', Human-readable failure description |
| scanned_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP, Server-side timestamp of validation — this is the legally meaningful moment |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, NOT NULL |

*Table 5.2.2.5: Data Dictionary and Description of Transactions Table*

- **Entity: Lease_Cycles**

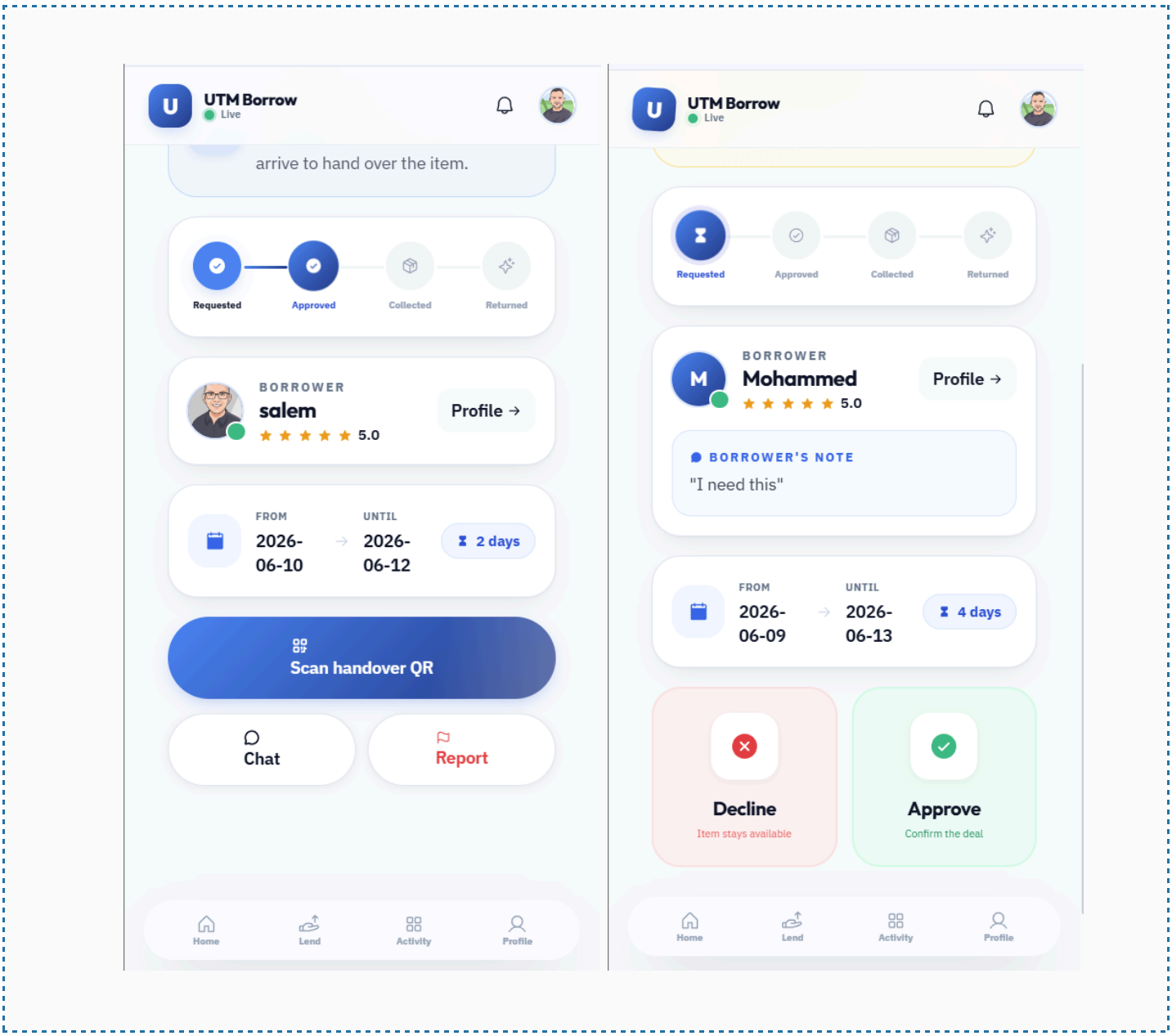
Tracks the active loan period, expected return date, and overdue status.

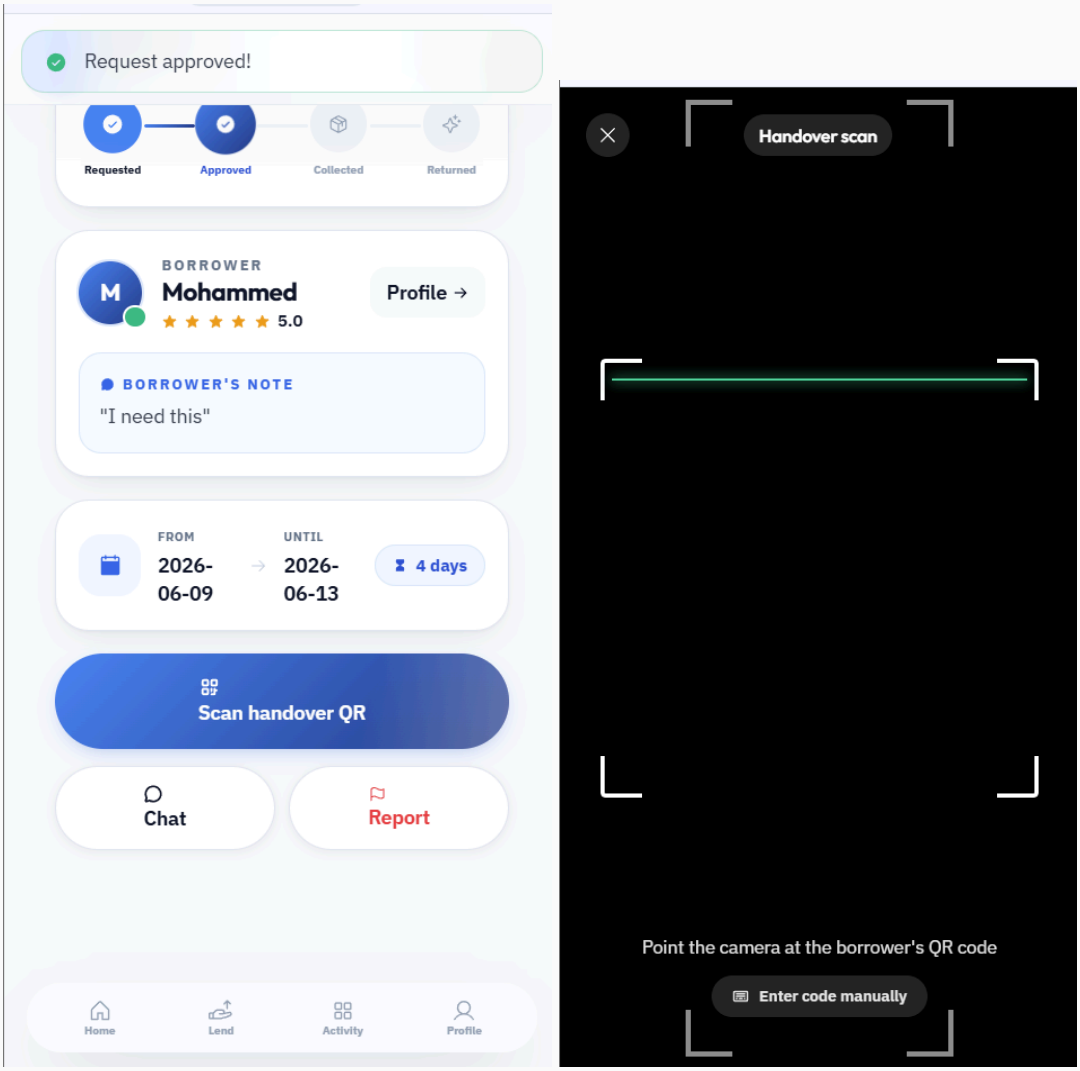
| Attribute name | Data Type | Description |
|---|---|---|
| lease_cycle_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| transaction_id | INT | NOT NULL, UNIQUE, FOREIGN KEY → transactions.transaction_id, ON DELETE CASCADE, Enforces one lease lifecycle per transaction |
| handover_scan_event_id | INT | NULLABLE, FOREIGN KEY → scan_events.scan_event_id, ON DELETE SET NULL, References the successful handover scan; NULL until handover occurs |
| return_scan_event_id | INT | NULLABLE, FOREIGN KEY → scan_events.scan_event_id, ON DELETE SET NULL, References the successful return scan; NULL until return occurs |
| handover_timestamp | TIMESTAMP | NULLABLE, Copied from the associated successful handover Scan_Event for query performance |
| return_timestamp | TIMESTAMP | NULLABLE, Copied from the associated successful return Scan_Event for query performance |
| expected_return_date | DATE | NOT NULL, Mirrored from transactions.borrow_end_date at lease creation |
| lease_status | ENUM | NOT NULL, DEFAULT 'Awaiting_Handover', Options: 'Awaiting_Handover', 'Active', 'Completed', 'Overdue' |
| overdue_days | INT | NOT NULL, DEFAULT 0, Computed value: MAX(0, CURRENT_DATE − expected_return_date) when lease_status = 'Active' or 'Overdue' |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, NOT NULL, Lease record creation (typically at transaction approval) |
| updated_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP |

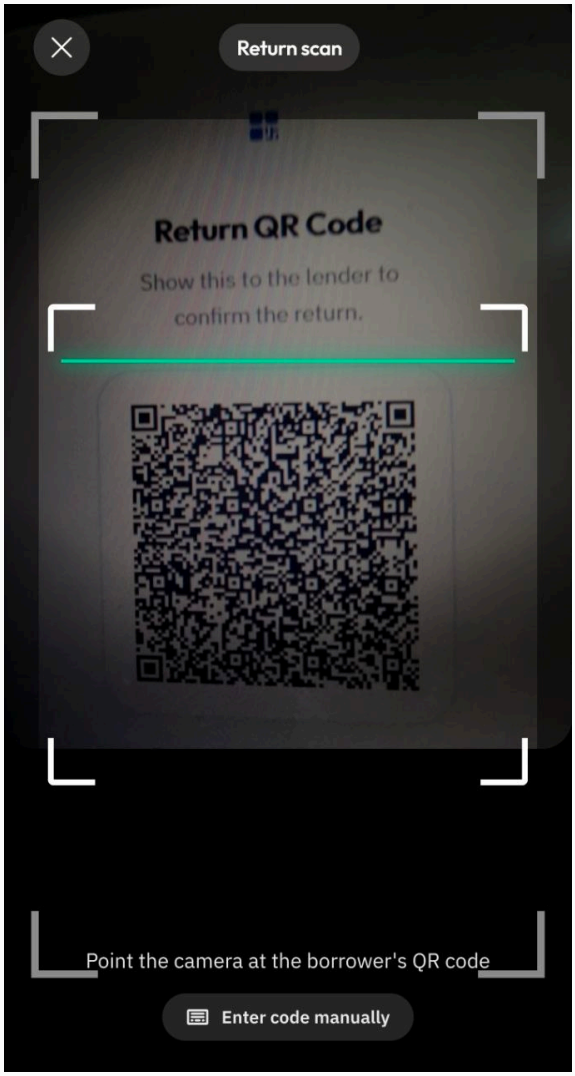
*Table 5.2.2.6: Data Dictionary and Description of QR_Tokens Table*

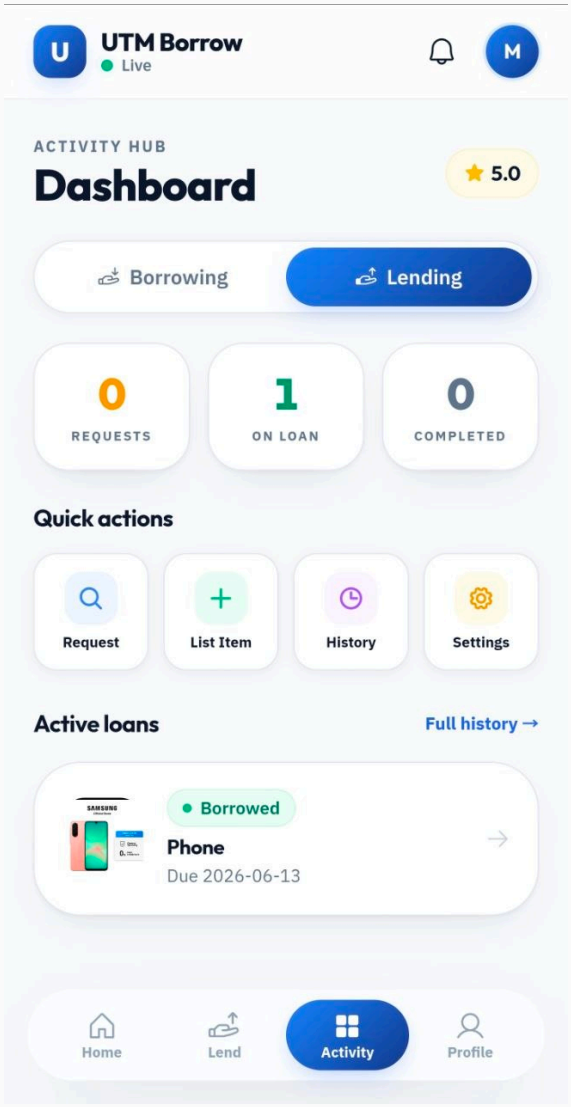
### 5.2.3. User Interface Design

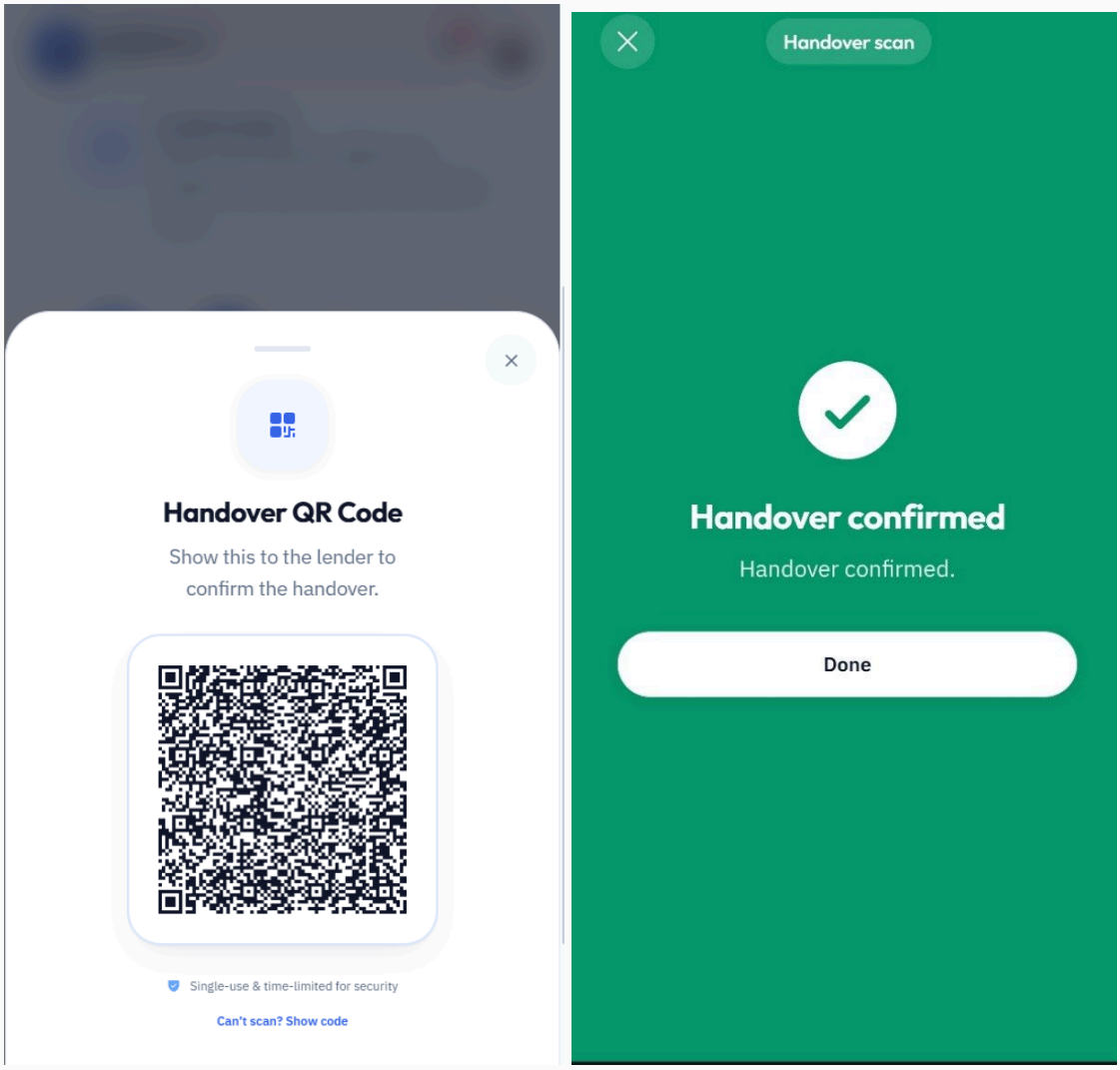
The screens below show the borrower's QR display, the lender's scanner for handover and return, the success confirmation, and a scan-failure (invalid token) state.




Request approved!
Requested
Approved
Collected
Returned
BORROWER
Mohammed
5.0
Profile →
BORROWER'S NOTE
"I need this"
FROM
2026-06-09
UNTIL
2026-06-13
4 days
Scan handover QR
Chat
Report
Home
Lend
Activity
Profile
Handover scan
Point the camera at the borrower's QR code
Enter code manually


Return scan
Return QR Code
Show this to the lender to confirm the return.
Point the camera at the borrower's QR code
Enter code manually


UTM Borrow
Live
M
ACTIVITY HUB
Dashboard
5.0
Borrowing
Lending
0
REQUESTS
1
ON LOAN
0
COMPLETED
Quick actions
Request
List Item
History
Settings
Active loans
Full history →
Borrowed
Phone
Due 2026-06-13
Home
Lend
Activity
Profile


Handover QR Code
Show this to the lender to confirm the handover.
Single-use & time-limited for security
Can't scan? Show code
Handover scan
Handover confirmed
Handover confirmed.
Done

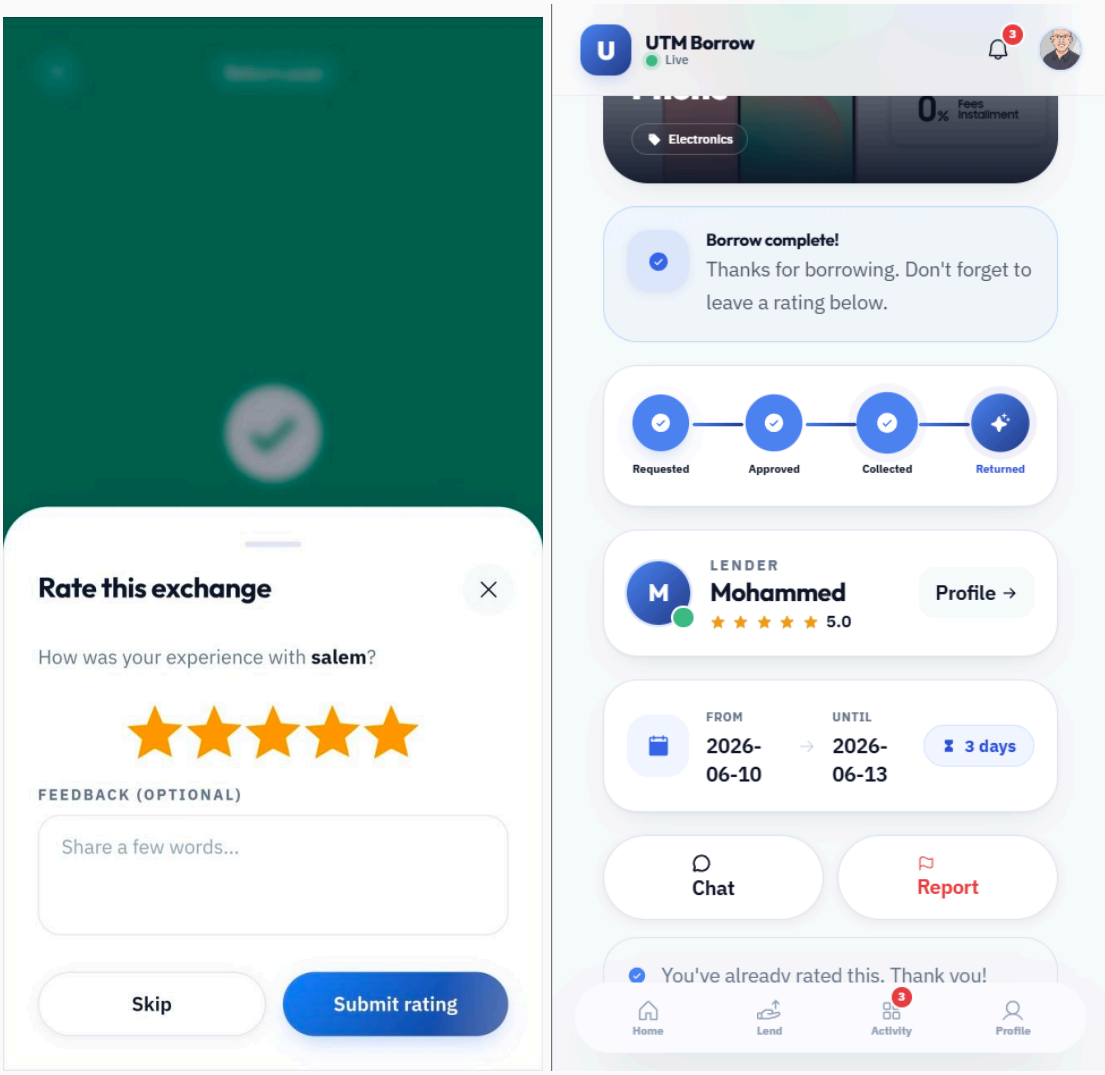



*Figure 5.2.3.1: Borrower QR Display (Handover) for QR VERIFICATION MODULE*

## 5.3. COMMUNITY MODERATION & REPORTING MODULE - Alsakkaf

This module keeps the platform safe. Any verified student can report an item or user; reports enter a queue for an MFA-elevated admin, who reviews full context and either dismisses the report or takes enforcement action — removing the item or suspending/banning the user. Every action is permanently audited.

### 5.3.1. Data Design

The module adds Reports, Admins, Moderation_Actions, and User_Suspensions, and extends Notifications. Reports reference the reporter, the reported item, and the reported user; enforcement writes a Moderation_Actions record and, for suspensions, a User_Suspensions record. Figure 5.3.1 shows these relationships.

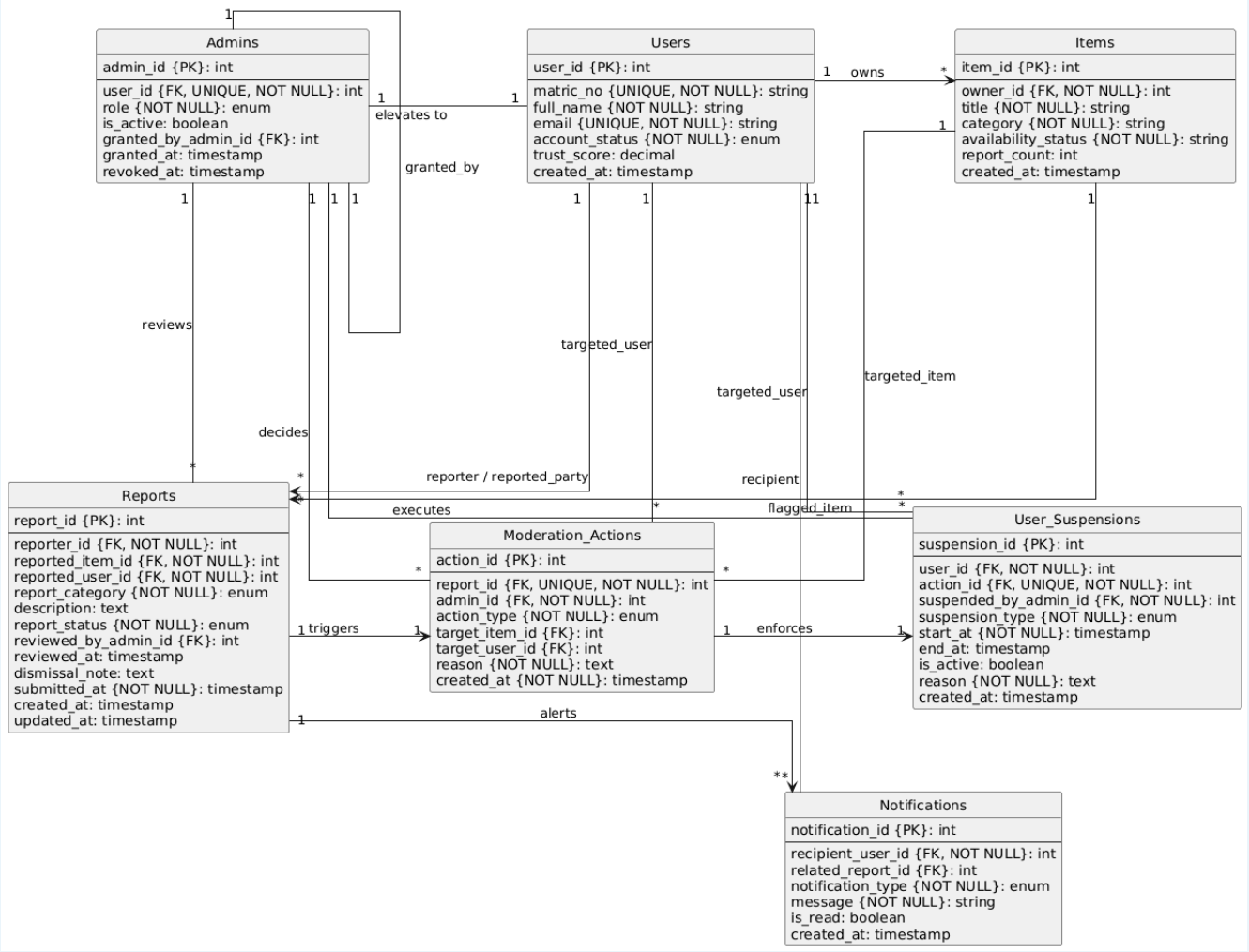



*Figure 5.3.1: Entity Relationship Design (ERD) for COMMUNITY MODERATION & REPORTING MODULE*

## 5.3.2. Data Dictionary & Description

The tables below define the moderation entities and the referenced Users and Items fields used in enforcement.

- **Entity: Users** **(Referenced — owned by Identity & User Management Subsystem)**

The account under review; its account_status is changed on suspension or ban.

| Attribute name | Data Type | Description |
|---|---|---|
| user_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| matric_no | VARCHAR(20) | NOT NULL, UNIQUE, UTM matriculation number |
| full_name | VARCHAR(100) | NOT NULL, Display name surfaced in moderation dashboard |
| email | VARCHAR(100) | NOT NULL, UNIQUE, Used for enforcement notifications |
| trust_score | DECIMAL(3,2) | DEFAULT 5.00, Range 0.00–5.00, Reviewed by admin during enforcement decisions |
| account_status | ENUM | Updated by the moderation service when a suspension or ban is applied or lifted. |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

*Table 5.3.2.1: Data Dictionary and Description of Users Table*

- **Entity: Items** **(Referenced — owned by Identity & User Management Subsystem)**

The reported listing; its availability_status becomes Removed on takedown.

| Attribute name | Data Type | Description |
|---|---|---|
| item_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| owner_id | INT | NOT NULL, FOREIGN KEY → users.user_id, Identifies the lender (subject of reports) |
| title | VARCHAR(150) | NOT NULL, Item display name surfaced in admin queue |
| category | VARCHAR(50) | NOT NULL, Used in moderation context display |
| availability_status | VARCHAR(20) | NOT NULL, DEFAULT 'Available', Transitions to 'Removed' on enforcement action, hidden from catalog when status = 'Removed' |
| report_count | INT | NOT NULL, DEFAULT 0, Denormalized counter incremented atomically on each new report — used for priority sorting in admin queue |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP |

*Table 5.3.2.2: Data Dictionary and Description of Items Tablee*

- **Entity: Admins**

Moderator accounts and their role/authority.

| Attribute name | Data Type | Description |
|---|---|---|
| admin_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| user_id | INT | NOT NULL, UNIQUE, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, One-to-one link with a base User account |
| role | ENUM | NOT NULL, DEFAULT 'Moderator', Options: 'Moderator', 'Senior_Moderator', Determines enforcement authority scope |
| is_active | BOOLEAN | NOT NULL, DEFAULT TRUE, FALSE indicates revoked moderator status without losing historical records |
| granted_by_admin_id | INT | NULLABLE, FOREIGN KEY → admins.admin_id, ON DELETE SET NULL, Establishes chain-of-trust for moderator appointments |
| granted_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP, Moderator privilege grant timestamp |
| revoked_at | TIMESTAMP | NULLABLE, Populated when is_active transitions to FALSE |

*Table 5.3.2.3: Data Dictionary and Description of Admins Table*

- **Entity: Reports**

A flagged item or user moving through the report lifecycle.

| Attribute name | Data Type | Description |
|---|---|---|
| report_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| reporter_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, The student submitting the report |
| reported_item_id | INT | NOT NULL, FOREIGN KEY → items.item_id, ON DELETE RESTRICT, The flagged item |
| reported_user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, Denormalized from items.owner_id at report submission — preserves lender identity at moment of flagging |
| report_category | ENUM | NOT NULL, Options: 'Prohibited_Illegal', 'False_Scam', 'Damaged_Dangerous', 'Inappropriate_Offensive', |

| | | |
|---|---|---|
| | | 'False_Listing', 'Other', Validated at submission (SRS UC3301 A1) |
| description | TEXT | NULLABLE, Optional narrative — max 500 characters enforced at app layer |
| report_status | ENUM | NOT NULL, DEFAULT 'Pending', Options: 'Pending', 'Under_Review', 'Dismissed', 'Actioned', State machine governing report lifecycle |
| reviewed_by_admin_id | INT | NULLABLE, FOREIGN KEY → admins.admin_id, ON DELETE SET NULL, Set when admin opens the report for review |
| reviewed_at | TIMESTAMP | NULLABLE, Set when report_status transitions to 'Dismissed' or 'Actioned' |
| dismissal_note | TEXT | NULLABLE, Populated only when report_status = 'Dismissed', Admin rationale for dismissal |
| submitted_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP, Original report submission time |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, NOT NULL |
| updated_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, ON UPDATE CURRENT_TIMESTAMP |

*Table 5.3.2.4: Data Dictionary and Description of Reports Table*

- **Entity: Moderation_Actions**

The immutable record of each enforcement decision.

| Attribute name | Data Type | Description |
|---|---|---|

| | | |
|---|---|---|
| action_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| report_id | INT | NOT NULL, UNIQUE, FOREIGN KEY → reports.report_id, ON DELETE CASCADE, Enforces strict one-action-per-resolved-report policy |
| admin_id | INT | NOT NULL, FOREIGN KEY → admins.admin_id, ON DELETE RESTRICT, Attributes the enforcement decision to a specific moderator |
| action_type | ENUM | NOT NULL, Options: 'Dismiss', 'Remove_Item', 'Suspend_User', 'Permanent_Ban', Determines side-effect propagation |
| target_item_id | INT | NULLABLE, FOREIGN KEY → items.item_id, ON DELETE SET NULL, Populated when action_type = 'Remove_Item' |
| target_user_id | INT | NULLABLE, FOREIGN KEY → users.user_id, ON DELETE SET NULL, Populated when action_type IN ('Suspend_User', 'Permanent_Ban') |
| reason | TEXT | NOT NULL, Mandatory admin justification — no enforcement action may be persisted without recorded rationale |
| created_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP, Immutable enforcement timestamp |

*Table 5.3.2.5: Data Dictionary and Description of Moderation_Actions Table*

- **Entity: User_Suspensions**

The duration and status of each suspension/ban.

| Attribute name | Data Type | Description |
|---|---|---|
| suspension_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE RESTRICT, The suspended user |
| action_id | INT | NOT NULL, UNIQUE, FOREIGN KEY → moderation_actions.action_id, ON DELETE RESTRICT, Links each suspension to its originating enforcement action |
| suspended_by_admin_id | INT | NOT NULL, FOREIGN KEY → admins.admin_id, ON DELETE RESTRICT, The admin who issued the suspension |
| suspension_type | ENUM | NOT NULL, Options: '3_Day', '7_Day', '30_Day', 'Permanent', Drives end_at computation |
| start_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP, Suspension commencement moment |

| | | |
|---|---|---|
| end_at | TIMESTAMP | NULLABLE, Computed as start_at + suspension_type duration; NULL when suspension_type = 'Permanent' |
| is_active | BOOLEAN | Set to FALSE automatically by the server when a timed suspension expires (checked on the user's next login). |
| reason | TEXT | NOT NULL, Mirrored from parent moderation_action for query convenience |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, NOT NULL |

*Table 5.3.2.6: Data Dictionary and Description of User_Suspensions Table*

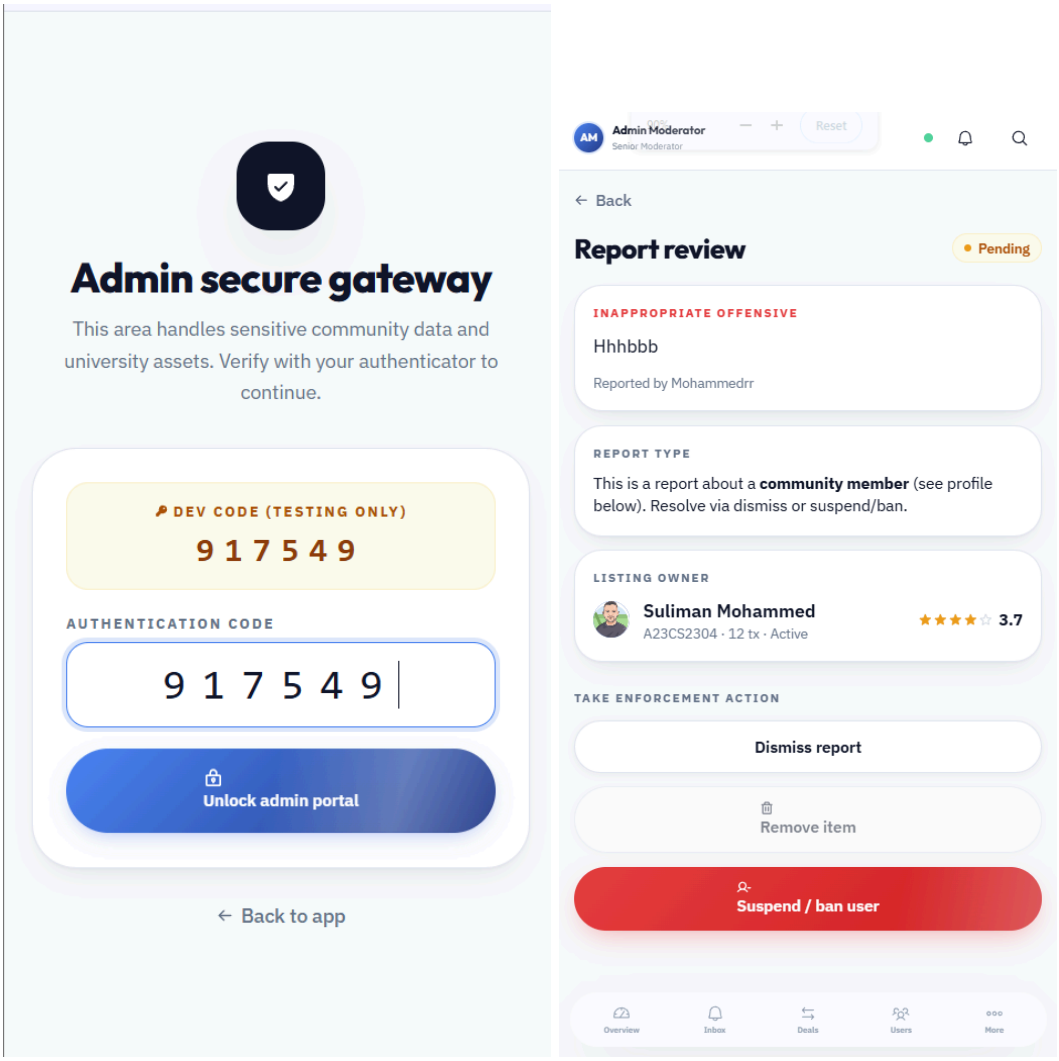
- **Entity: Notifications** **(Extended from Module 1)**

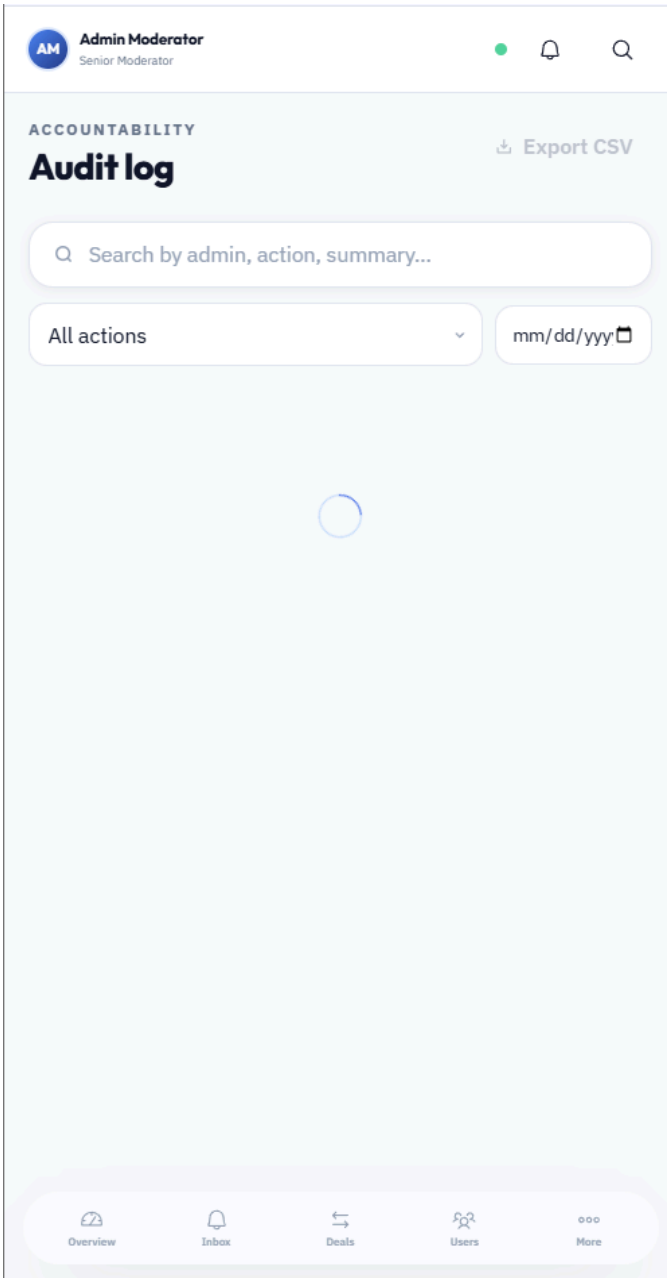
Moderation-specific alerts to reporters and affected users.

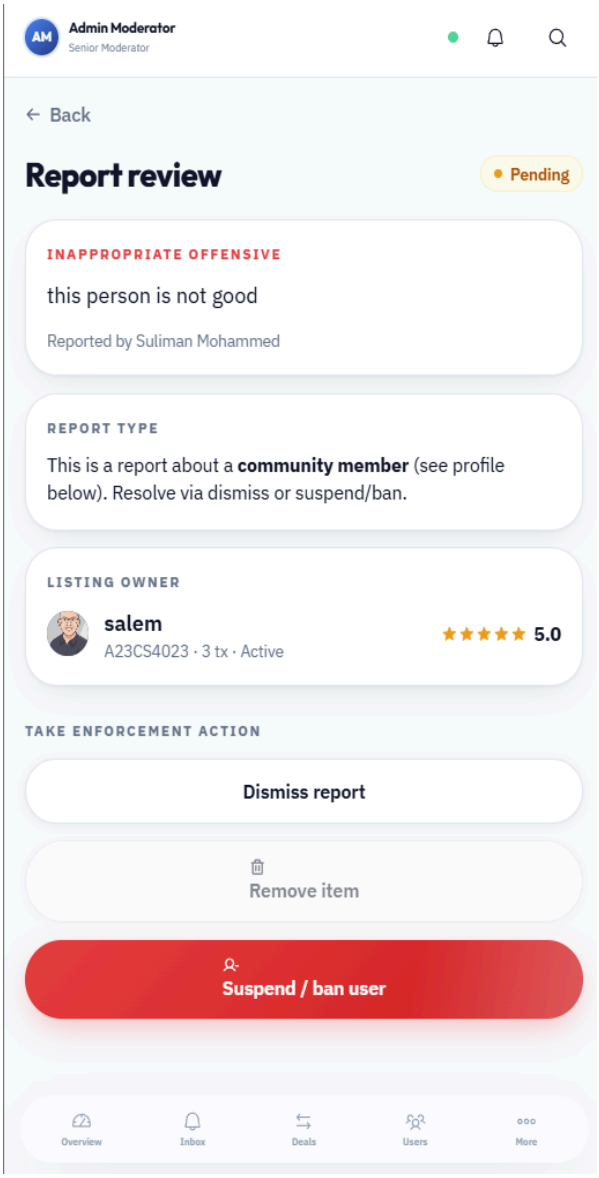
| Attribute name | Data Type | Description |
|---|---|---|
| notification_id | INT | PRIMARY KEY, AUTO_INCREMENT |
| recipient_user_id | INT | NOT NULL, FOREIGN KEY → users.user_id, ON DELETE CASCADE, The notification recipient |
| related_report_id | INT | NULLABLE, FOREIGN KEY → reports.report_id, ON DELETE SET NULL, Contextual report reference when applicable |
| notification_type | ENUM | NOT NULL, Options: 'Report_Submitted', 'Report_Reviewed', 'Item_Removed', 'Account_Suspended', 'Account_Reinstated', Moderation-specific notification taxonomy |
| message | VARCHAR(255) | NOT NULL, Pre-formatted human-readable alert text |
| is_read | BOOLEAN | DEFAULT FALSE, Toggled to TRUE when user opens the notification |
| created_at | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP, Dispatch time |

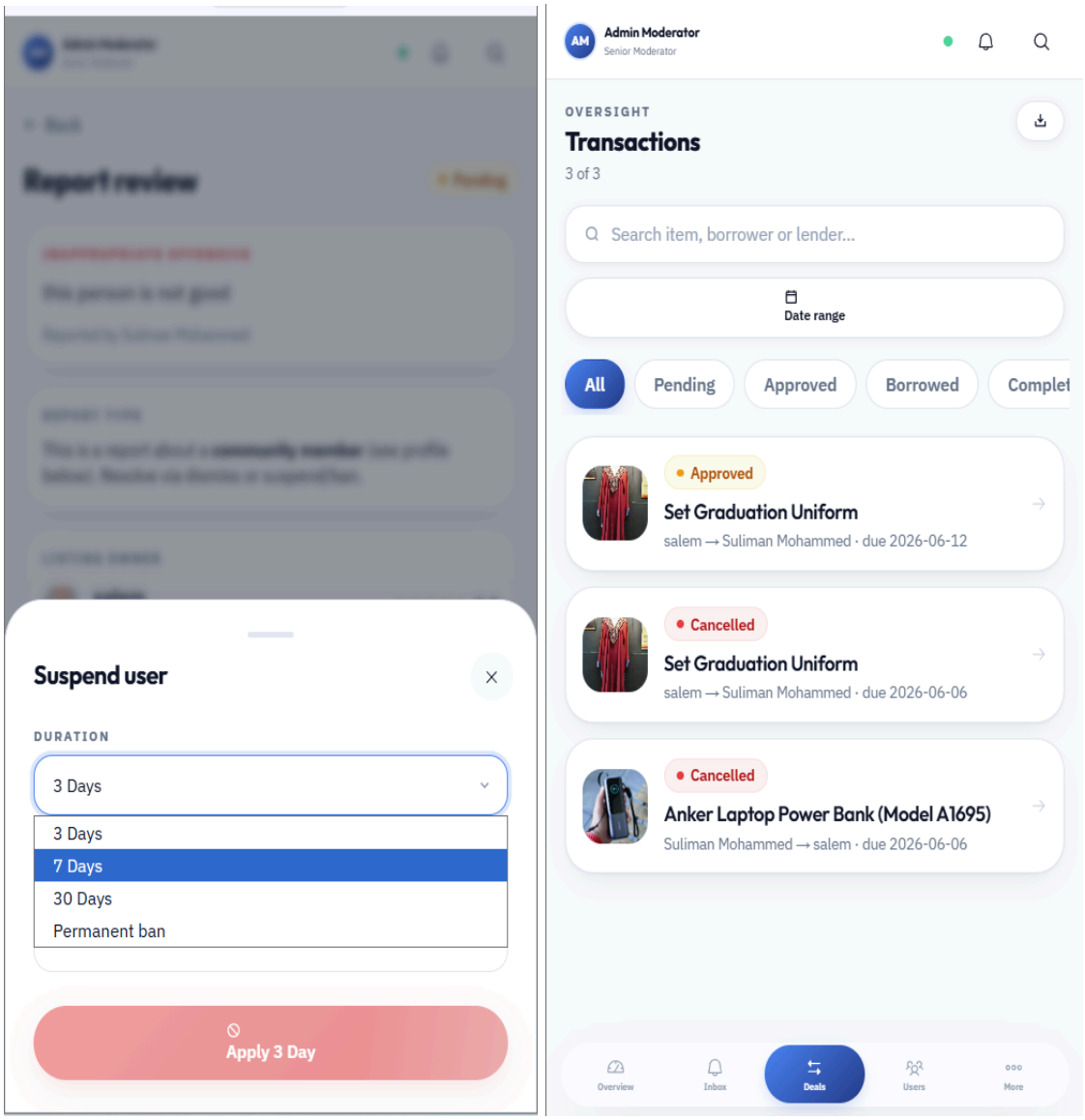
*Table 5.3.2.7: Data Dictionary and Description of Notifications Table*

## 5.3.3. User Interface Design




Admin Moderator
Senior Moderator
ACCOUNTABILITY
Audit log
Export CSV
Search by admin, action, summary...
All actions
mm/dd/yyyy
Overview
Inbox
Deals
Users
More


Admin Moderator
Senior Moderator
Back
Report review
Pending
INAPPROPRIATE OFFENSIVE
this person is not good
Reported by Suliman Mohammed
REPORT TYPE
This is a report about a community member (see profile below). Resolve via dismiss or suspend/ban.
LISTING OWNER
salem
A23CS4023 · 3 tx · Active
5.0
TAKE ENFORCEMENT ACTION
Dismiss report
Remove item
Suspend / ban user
Overview
Inbox
Deals
Users
More


Suspend user
DURATION
3 Days
3 Days
7 Days
30 Days
Permanent ban
Apply 3 Day
Admin Moderator
Senior Moderator
OVERSIGHT
Transactions
3 of 3
Search item, borrower or lender...
Date range
All
Pending
Approved
Borrowed
Approved
Set Graduation Uniform
salem → Suliman Mohammed · due 2026-06-12
Cancelled
Set Graduation Uniform
salem → Suliman Mohammed · due 2026-06-06
Cancelled
Anker Laptop Power Bank (Model A1695)
Suliman Mohammed → salem · due 2026-06-06
Overview
Inbox
Deals
Users
More